

has been explained in section 5.4.2) of the product, can improve customer satisfaction to a large extent. From this perspective, the following metrics are important:

- Fix backlog and backlog management index
- Fix Response time
- Percent delinquent fixes (or the percentage adherence to the Service Level Adherence)
- Fix quality (as judged by "Bad Fix")

5.5.3.1 Fix Backlog and Backlog Management Index

Fix backlog is a workload statement for software maintenance. It is related to both the rate of defect arrivals and the rate at which fixes for reported problems become available. It is a simple count of reported problems that remain opened at the end of each period (month or week). By keeping the data in the form of trend charts, this metric can provide meaningful information for managing the maintenance process.

Another metric to manage the backlog of open problems is the backlog management index (BMI).

$$\text{BMI} = \frac{\text{Number of problems closed during the month}}{\text{Number of problem arrivals during the month}} \times 100$$

Since the BMI is a ratio as defined, a BMI larger than 100, it means the backlog is reduced. If BMI is less than 100, then backlog increases. The goal is always to strive for a BMI larger than 100.

5.5.3.2 Fix Response Time

For many organizations, guidelines are established on the time limit within which the fixes should be available for the reported defects. Usually the criteria are set in accordance with the severity of the problems:

1. (most severe) – Data loss, hardware damage, or a safety related failure
2. – Loss of functionality *without* any reasonable workaround
3. – Loss of data or functionality with a reasonable workaround
4. – Partial loss of a function or a feature set
5. (least severe) – A cosmetic error

For "Crit-Sits (Critical Situations) where the customers' businesses are at risk due to the defects encountered in the software product, teams work round the clock to fix the problems. For less severe defects, for

which workarounds are available, there are more relaxed *fix response time* requirements. The fix response time metric is usually calculated as follows for all problems as well as by severity level:

Mean time of all problems from open to close

If data points collected have extreme values, medians should be used instead of mean. Such cases could occur for less severe problems for which customers may be satisfied with the workaround and did not demand a final fix. Therefore the problem may remain open for a long time in the defect tracking report.

5.5.3.3 Percent Delinquent Fixes

Another measure for sensitive metric in maintenance phase is the percentage of delinquent fixes. For each fix, if the turnaround time exceeds the response time by severity, then it is classified as "delinquent":

Percent delinquent fixes =

$$\frac{\text{Number of fixes that exceeded the fix response time criteria by severity level}}{\text{Total number of fixes delivered in a specified time}} \times 100$$

This metric is related to the well known as the *percent adherence* to the SLA (service level agreement), which is defined only slightly differently:

$$\frac{\text{Number of fixes provided within the SLA time specified (as per the severity level)}}{\text{Total number of fixes delivered in a specified time}} \times 100$$

5.5.3.4 Fix Quality

Fix quality or the number of defective fixes is another important quality metric for the maintenance phase. From the customers' perspective, it is bad enough if they have to encounter functional defects when they are running their business using the software. It is even worse if the fixes have to be turned down due to some defect in them. As discussed earlier, for mission-critical software applications, defective fixes adversely affect customer satisfaction. The metric of percent defective fixes is simply the percentage of all fixes (in a given time interval) that turn out to be defective. A fix is defective if it did not fix the problem reported by the customer, or if it fixed the original problem but injected a new problem (that is why regression testing is so important. *Regression testing* is briefly discussed in Chapter 10).

"Bad Fix" or "Defective Fix" is the other commonly used term to express the quality of fix. Bad fixes include:

- Number of fixes rejected by the customer
- Bad fix injection

Bad fix injection(s) are the new defect(s) that are introduced when performing a service request (Fixing a problem). Bad fix percentage is calculated as:

% Bad Fix =

$$\frac{(\text{Number of Rejected Fixes}) + (\text{Number of New Defects introduced})}{(\text{Total Number of fixes delivered over a specified period of time})} \times 100$$

It is at this point that it becomes possible to carry out retrospective analyses of the *defect removal efficiencies* of each specific review, inspection and test and of the cumulative efficiency of the overall series of defect removal steps.

Figure 6 below summarizes application of measures for process management.

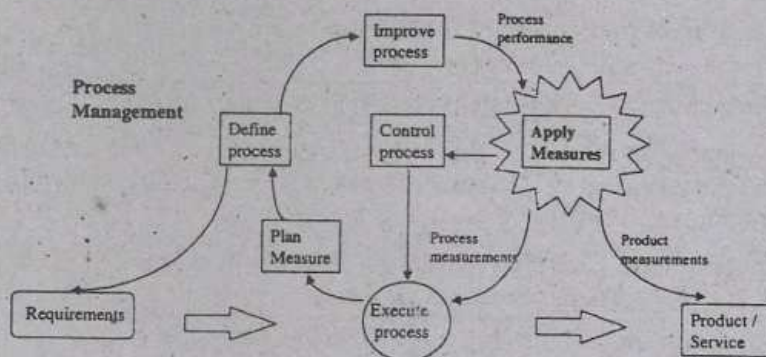


Figure 6 Applying Measures to Process Management

5.6 Classification of Software Metrics

According to the Software Engineering Institute (SEI—[2], software metrics may be broadly classified as either *product metrics* or *process metrics*. *Product metrics* are measures of the software product at any stage of its development, from requirements to installed system. Product metrics may measure the complexity of the software design, the size of the source code or the object code, or the number of pages of documentation produced. *Process metrics*, on the other hand, are measures of the software development process, such as overall development time, type of methodology used, the average level of experience of the development staff.

However, this is not the only way to classify metrics; i.e. further classification of Process Metrics and Product Metrics is possible. Both of them may be distinguished on their *objective* and from

subjective properties. Generally speaking, objective metrics should always result in identical values for a given metric, as measured by two or more qualified observers. For subjective metrics, even qualified observers may measure different values for a given metric, since their subjective judgment is involved in arriving at the measured value. As an example of this—for the product metrics, the SIZE of the product measured in lines of code (LOC) is an objective measure. An example of a subjective product metric is the classification of the software as “organic”, “semi-detached” or “embedded” as required in the COCOMO (Constructive Cost Model) cost estimation model proposed by Boehm. As another example—for process metrics, development time is an example of an objective measure, whereas the level of developer experience is likely to be a subjective measure.

Yet another way in which metrics can be categorized is a *primitive* (or *primary*) metrics or *computed* (or *derived*) metrics. Primitive (or Primary) metrics are those that can be directly observed (such as program size in LOC, number of defects observed in unit testing or total development time for the project). Computed (or Derived) metrics are those that cannot be directly observed but are computed in some manner from other metrics. For example—metrics commonly used for productivity, such as LOC produced per person-month (LOC/person-month), or for product quality, such as the number of defects per thousand lines of code (defects/KLOC). Computed metrics are combination of other metric values and thus are often more valuable in understanding or evaluating the software process than are simple metrics.

Having discussed classification of metrics, the next section, is devoted to requirements related metrics.

5.7 Requirements Related Metrics

Readers may wonder why the topic of “Requirements related metrics” is discussed as a separate section. There are two good reasons for this. First of all, requirements have an all-encompassing relationship to other project processes as depicted in figure 7 below. It shows the linkage between functional requirements to the software products including design and code. As you’ll see later in this section, the requirements related metrics help us report on the percentage of requirements traced.

The second reason comes from the fact that the new Integrated Capability Maturity Model (known as the CMM-I) focuses on this as shown in the vignette on the next page. (In Chapter 9, the CMM-I model has been discussed as among other process improvement frameworks). The Integrated CMM Model expects what is known as “bi-directional traceability” of requirements.

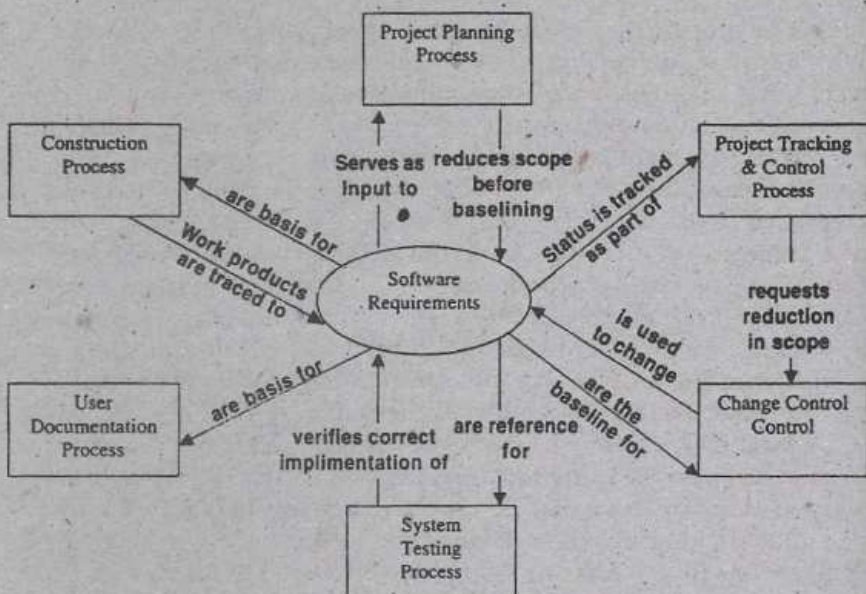


Figure 7: Relationship of requirements to other project processes

Requirements Related Metrics: the CMM Integrated Perspective

Level 2: Process Area-REQUIREMENTS MANAGEMENT

The CMM Integrated model expects that the organization is able to:

Monitor and control the requirements management process against the plan for performing the process and take appropriate corrective action.

Level 3: Process Area-REQUIREMENTS DEVELOPMENT

The CMM Integrated model expects that the organization is able to:

Monitor and control the requirements development process against the plan for performing the process and take appropriate corrective action.

This means that a good amount of control on requirements is expected. For this, there should be requirements related metrics in place. *Requirements metrics* pertain to specification, translation and volatility of requirements. The two well-known requirements metrics are—*Requirements Traceability* and *Requirements Stability Index*. Both of them pertain to *functional requirements*. The vignette on the next page explains what functional requirements are. Next, we discuss further details on requirements related metrics.

Levels of Requirements

Software requirements include three distinct levels of requirements: business requirements, user requirements and functional requirements.

Business requirements represent high-level objectives of the organization or customer requesting the system or product.

User requirements describe the tasks the user must be able to accomplish with the product.

Functional requirements define the software functionality the developers must build into the product to enable users to accomplish their tasks thereby satisfying the business requirements.

5.7.1 Requirements Traceability

This traces the linkage between functional requirements and the software product including design & code and reports on the percentage of requirements traced from the function to the test case. The table below shows recommended items for requirements traceability metric tracking.

Table 1: Recommended items for requirements traceability metric tracking

From	To	Backward trace also ?
User requirements (mission)	User requirements (system)	Yes
User requirements (system)	System requirements	Yes
System requirements	Software requirements	Yes
Software requirements (SRS)	Software design high level	Optional
Software requirements (SRS)	Software design detailed	Optional
Software requirements (SRS)	Code	Optional
Software requirements (SRS)	Software qualification test cases	Optional

Note: SRS is Software Requirements Specification

5.7.2 Requirements Stability Index

This tracks changes to the functional requirements and plots the cumulative number of changes requests over time against the number of requests that have been resolved. Thus, a requirement stability index (RSI) is a metric used to organize, control, and track changes to the originally specified requirements for a new system project or product.

Typically, a project begins, after consultation with customers or clients and research into their needs, with the creation of a *requirements document*. The document expresses what the customer or client needs and expects and, at least implicitly, what the developer will provide.

The client or customer representative group reviews the document and, if in agreement with its specifications, signs it. This process (called *signing off*) is intended to ensure that customer representatives or clients have agreed—in writing—on the specifics involved.

Almost inevitably, however, once the design and development process is underway, customers or clients think of changes or embellishments they would like, a phenomenon known as the phenomenon of "requirements creep" or "feature creep". This, if not managed with a firm hand, requirements/feature creeps can result in lost time and money and either a project far beyond the scope of what was originally foreseen, or a failed project.

Figure 8 shows an example of the total number of requirements, the cumulative number of requirements changes, and the number of remaining TBDs over time. It may be desirable to also show the number of added, modified and deleted requirements over time.

As a concluding remark, we note that requirements stability is related to size stability. This is because, size stability, which is derived from changes in the size estimate as time goes on, provides an indication of the completeness and stability of the requirements, the understanding of the requirements, design thoroughness and stability, and the capability of the software development staff to meet the current budget and schedule. Size instability may indicate the need for corrective action.

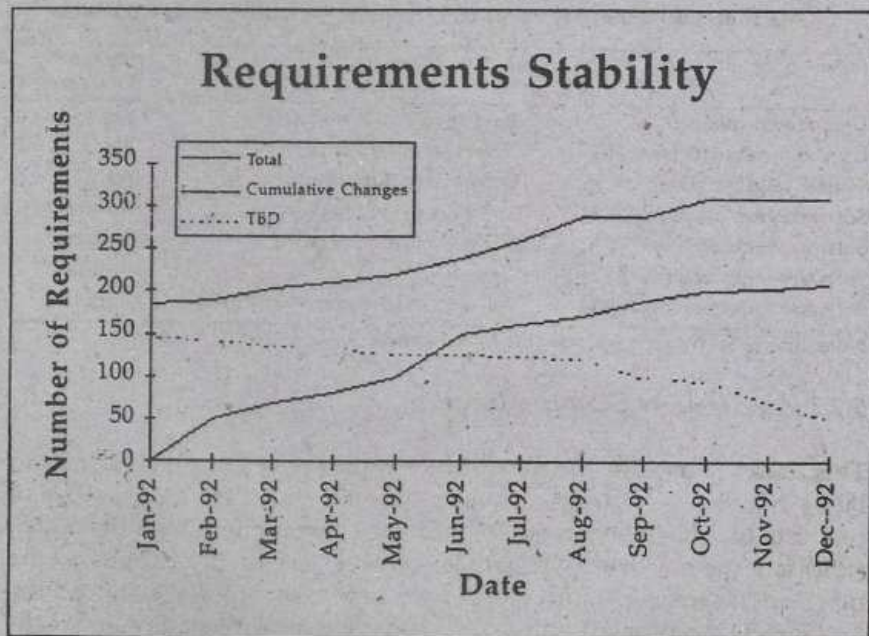


Figure 8: Requirements Stability Indicator

5.8 Measurements and Process Improvement

Having discussed the role of measurement in the software life cycle and classification of software metrics, let us now discuss the role of metrics in process improvement. Process Improvement is a cyclical change process (depicted in Figure 9) in which particular process changes are introduced in order to remedy specific process defects in an environment of continuous process evolution. In this context, measurement is a means of assisting diagnosis of process problems, and of monitoring the effect of process change. Measurement, by itself, will not tell you what changes you need to make in order to improve your process. However, it can help to identify the most significant areas of weakness (even in a 'healthy' process) and allow you to decide whether a process change has the desired effect.

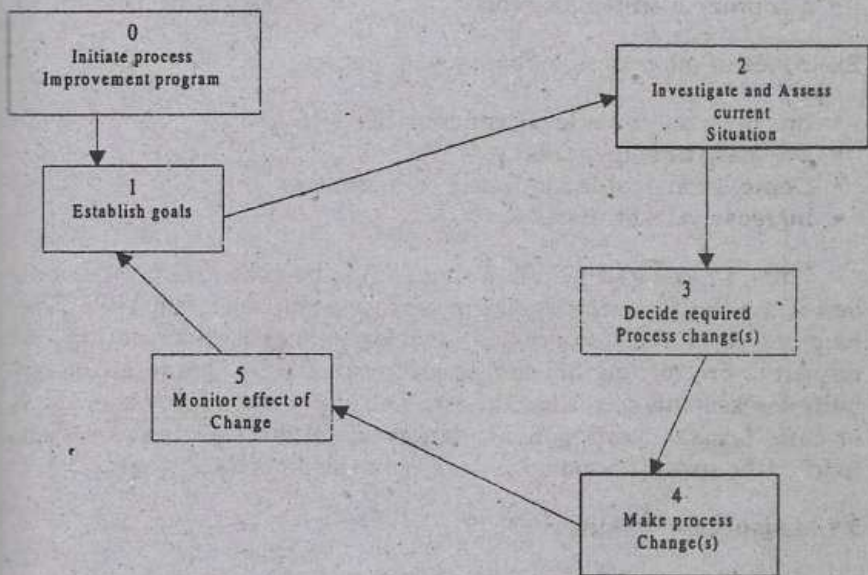


Figure 9: Process Improvement Cycle

As shown in figure 9 above, process improvement can be viewed as a cyclical set of activities consisting of 5 main stages:

1. Establishing the strategic goals of a process improvement activity
2. Investigating the current situation to set tactical goals for process improvement
3. Investigating process change options that could deliver the required improvements
4. Making the agreed process change(s)
5. Monitoring the effects of the change(s) to assess whether the desired process improvement has been delivered.

After stage 5, the cycle starts all over again, if necessary. However, before an organization can initiate a rolling cycle of process improvement, it must be clear on what its current stage is and where it wants to reach i.e. its ultimate target for a cycle. This is where data collected (from process observations) in form of measurement helps. Thus, it is important to appreciate that it is difficult to define process improvement goals without the real idea of what your problems are. *If you don't know where you are, it's pretty difficult to work out how to get where you want to be.*

Let us consider some examples of goals. For example, here are a number of goals, which relate directly to the software maintenance process:

- Maximize customer satisfaction
- Make maintenance work more predictable
- Minimize maintenance cost

Examples of other types of business goals are:

- Improve software development efficiency
- Decrease time to market
- Demonstrate value for money to customers
- Increase market share

Thus, we can say that measurement can be used to support a number of the individual activities in a process improvement cycle. They help you to establish the current development process capability with respect to product quality and project productivity. Quantitative capability assessments can be used to establish high-level improvement goals or to undertake detailed investigations of software processes in order to identify areas of maximum leverage to achieve those goals.

5.9 Measurement Principles

5.9.1 Need for Measurement Principles

In this section, we discuss the technical issues associated with software measurement and the types of measures that are available to support the goals of an organization. Once you have defined your metrics requirements (by planning out a process improvement cycle) you may find you need to obtain metrics your organization does not normally collect. In such cases, you will need to know whether existing measures meet your requirements and, if they do not, how you can adapt metrics or develop them yourself. This is the most difficult and controversial aspect of software metrics. For this reason, in this section, we review, in some detail, the theoretical aspects of what we mean by measurement. This provides the context for the discussion of practical data collection issues covered later in this chapter.

5.9.2 Structural Model for Measurement

We'll first introduce a "structural model for software measurement" (Figure 10) for our conceptual clarity. Then we'll give some examples of what can be measured and what are the issues to deal with in the real world scenario of software projects.

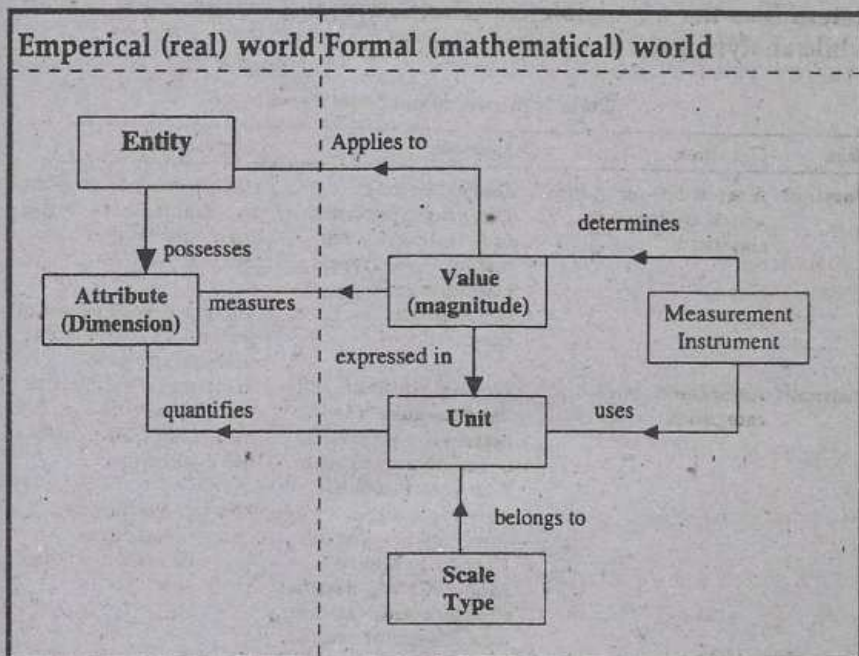


Figure 10: A Structural Model for Software Measurement

Attributes are the properties that an entity possesses. For a given attribute, there is a relationship of interest in the "real" world that we want to capture formally in the mathematical world. For example, if we observe two people, we can say that one is taller than the other. A measure of height allows us to capture the "is taller than" relationship and map it to a formal system, enabling us to explore the relationship mathematically. As Figure 10 above suggests, an entity possesses many attributes, while an attribute can qualify many different entities. For example, consider a computer program to be a software entity, which can exhibit attributes such as *length*, *structure* and *correctness*.

A measurement unit determines how we measure an attribute. The model depicted in Figure 10 above implies that an attribute may be measured in one or more units. The term 'metric' is often used for software units. For example, code length might be measured by counting the lines of code or the lexical tokens in a program listing.

5.9.3 Measurement Scales

When we consider *measurement units*, we need to understand the different measurement scale types implied by the particular units. The most common scales are: **nominal**, **ordinal**, **interval** and **ratio**. (See the Table 2 below on "Measurement Scale Types") A unit's scale type determines the admissible transformation and statistics we can use while analyzing the measures collected.

Table 2: Measurement Scale Types

Name	Definition	Examples	Constraints
Nominal	A set of categories into which an item is classified	Testing Methods: Designs Inspections, unit Testing, System testing, Error types: interface, I/O, computation, control flow	Categories cannot be used in formulae even if you map your categories to the integers. You can use the mode and percentiles to describe nominal datasets.
Ordinal	An ordered set of categories	Ordinal scales are often used for adjustment factors in cost models based on a fixed set or <i>scale points</i> such as very high, high, average, low, very low. The SEI Capability Maturity Model (CMM) classifies development/maturity on a five-point ordinal scale	Scale points cannot be used in formulae: so 2.5 on the SEI CMM scale is not meaningful. You can use medians and percentiles to describe ordinal dataset
Interval	Numerical values where the difference between each consecutive pair of numbers is an equivalent amount but there is no 'real' zero value. On an interval scale $2-1 = 4-3$ but 2 units are not twice as much as 1 unit	If you have been recording information at six-monthly intervals since 1990, you can measure time since the start of the measurement program on an interval scale starting with 01/01/1990 as 0, followed by 01/06/1990 as 1, and 01/01/1991 as 2 and so on. Degrees Fahrenheit and Celsius are interval scale measurement of temperature	
Ratio	Similar to interval scale measure but including an absolute value zero. On a ratio scale, 2 units are equivalent to twice the amount of 1 unit.	The number of lines of code in a program is a ratio scale measure of code length. Degrees Kelvin is a ratio scale measure of temperature	You can use the mean and standard deviation to describe ratio scale datasets

A measure is a mapping from the attribute box to the value box as shown in Figure 10. When we measure an attribute, we do so by applying a specific measurement unit to a particular entity to obtain a value. This value is often numerical, but it does not have to be. For example, a code module can be labeled 'inspected' or 'not inspected', or a defect found in a code review can be categorized as 'requirements error', 'design error', 'code error' or 'documentation error'. A measured value cannot be interpreted unless we know to what entity it applies, what attribute it measures and in what unit.

5.9.4 Measurement Principles: Summary

Having discussed the essential principles of measurements, let us now summarize for a quick re-cap. From the discussion of the structure of measurement; we can identify a number of principles to follow when we define software measures:

- We need to *specify the entities* to which the measures apply. This is essential to ensure comparability of measures.
- We need to *define the attribute* we are measuring. This is essential to ensure that we can interpret and use our results.
- We need to *define the unit* we are using. This is essential to ensure that we understand the values we obtain.
- We need to *specify the measurement instrument* we are using. Instruments are nothing but mechanisms to collect measures.

In addition, we may also need to define a *measurement protocol*. Protocols are set of rules that help in bringing consistency in measurements. They help us measure a specific attribute for a specific entity consistently and repeatably, independent of the measurer and the environment in which measurements are done. To illustrate the importance of measurement protocols, consider this example: A protocol for measuring the height of adult humans in meters might be:

- The person must be standing (not bending over)
- Measurement must start at the ~~top~~ of the head (not from the tip of up-stretched arms)
- The person must ~~remove~~ his/her foot ware
- The person must ~~stand~~ on the soles of the feet (not on tiptoe)

This example makes it clear that a measurement protocol is not simply derived from a definition of the measurement unit or the attribute. It is associated with obtaining a measurement value and can therefore be defined only when you have decided to measure a specific attribute on a specific entity using a specific measurement unit using a suitable (an preferably a calibrated) instrument. Unless these protocols

are specified, it may not be possible to replicate the measurement and replication is the basis of scientific validation.

In view of above discussion some fundamental measurement principles emerge. A measurement process can be characterized by the following activities:

- **Formulation:** The derivation of software measures and metrics that is appropriate for the representation of the software that is being considered.
- **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
- **Analysis:** The computation of metrics and the application of mathematical tools.
- **Interpretation:** The evaluation of metrics results in an effort to gain insight into the quality of the representation
- **Feedback:** Recommendations derived from the interpretation of technical metrics transmitted to the software team.

This, in turn, results in the following associated principles:

- The objectives of measurement should be established before data collection begins
- Each technical metric should be defined in clear terms.
- Metrics should be derived on a basis that is valid for the software application domain (for example if a metrics is designed on the design principle of "cohesion" then, a metric that measures module cohesion should increase in value as the level of cohesion increases)
- Metrics should be tailored to best accommodate specific products and processes.

Collection and analysis are the next important activities in a metrication program of an organization. In this regard, the following guiding principles should be remembered:

- Whenever possible, data collection and analysis should be automated
- Valid statistical techniques should be applied to establish relationships between internal product attributes and external quality characteristics (for example: a possible correlation between the architectural complexity of a software product and the number of defects reported during the production use)
- Interpretative guidelines and recommendations should be established for each metric

Finally, as with any new initiative, management support, adequate funding, and adequate awareness etc. are crucial for successful implementation of metrics program in an organization.

5.10 Identifying Appropriate Measures and Metrics for Projects

In a project scenario, you will need to measure the five main dimensions of software production and maintenance—*COST, QUALITY, EFFORT, SIZE* and *SCHEDULE*. There is usually conflict and overlap between these dimensions, when software producers have to balance the requirements for cost, quality and timeliness within the constraints of the required functionality. Readers should note that the new *integrated CMM* (Capability Maturity Model discussed in Chapter 9) looks for "prioritization of the goals based on the project dimensions mentioned above. Attempts to control only dimension are likely to lead to imbalances in the other. Therefore the metrics to be collected should provide indicators that help track ongoing project progress, software products, and software development processes in a balanced way. As said in the beginning of this chapter, the term *indicator* is used to denote a representation of metric data that provides insight into an ongoing software development project or process improvement activity. The defined indicators shown in the table on the next page are consistent with the Software Engineering Institute's Capability Maturity Model (CMM).

Table 3 shows the recommended metrics set for a project. It shows the indicator categories, the management insight provided, and the specific indicators for recommended metrics.

From a measurement viewpoint, the *size* of the software product is an element, which influences all dimensions. It allows us to 'normalize' measures in each dimension to enable cross-project comparisons. The term 'normalize', in this context means converting a measure into a rate or percentage. As an example for how "normalization" helps interpret the data, consider this—suppose there are two programmers 'A' and 'B'. Suppose further, that as per past experience 'A' can write on an average about 150 lines of code per pay while 'B' can about 120 lines of code per day as an average. With this data can we really conclude, who is a more productive/better programmer? Perhaps not; for that, we'll need an additional piece of information—namely, the number of defect free lines delivered by each of them. With this information we can certainly normalize by comparing the two programmers on the basis of their percent defect free line of code delivered.

Similarly, as far as the *EFFORT* is concerned, each project will need to monitor itself against its designated monetary budget, but for comparison purposes, we also need to measure staff *EFFORT*. In most software projects, the main element of costs is staffing *COSTS*. Since there are problems using monetary values to compare projects, we use measures of staff effort as well as monetary costs to represent the cost dimensions.

As discussed in Chapter 4, there are two main facets of quality: *process quality* and *product quality*. In addition, there are two viewpoints on

Table 3: Recommended Metrics Set for a Project

Indicator Category	Management Insight Provided	Indicators
Progress	Provides information on how well the project is performing with respect to its schedule.	Actual vs. planned task completions Actual vs. planned durations
Effort	Provides visibility into the contributions of staffing on project costs, schedule adherence, and product quality.	Actual vs. planned staffing profiles
Cost	Provides tracking of actual costs against estimated costs and predicts future costs.	Actual vs. planned costs Cost and schedule variances
Review Results	Provides status of action items from life-cycle review.	Status of action items
Trouble Reports	Provides insight into product and process quality and the effectiveness of the testing.	Status of trouble reports Number of trouble reports opened, closed, etc. during reporting period
Requirements Stability	Provides visibility into the magnitude and impact of requirements changes.	Number of requirements changes/clarifications Distribution of requirements over releases
Size Stability	Provides insight into the completeness and stability of the requirements and into the ability of the staff to complete the project within the current budget and schedule.	Size growth Distribution of size over releases
Computer Resource Utilization	Provides information on how well the project is meeting its computer resource utilization goals/requirements.	Actual vs. planned profiles of computer resource utilization
Training	Provides information on the training program and staff skills.	Actual vs. planned number of personnel attending classes

product quality: the internal characteristics of the product such as structure and modularity, and the external characteristics of the product during operation, such a reliability and usability. In the light of the discussion so far and Table 3, we can say that minimum attributes of a project measure are:

- Total project COST
- Staff EFFORT for development and maintenance
- Other costs (tools, training, travel etc.)
- Project DURATION (calendar time)

- Post-release DEFECTS and the origins of those defects (each defect must be classified as to WHERE (in the product it is) FOUND and WHEN (in the process it was) INTRODUCED.
- Product SIZE
- COST of RE-WORK before and after product release
- PRODUCT CHARACTERISTICS in its operational environment (e.g. time between failures; time to correct defects; speed of execution; learning time; positive and negative responses to user/customer satisfaction surveys)

The Table 4 expands on the types of attributes that can be measured in each of the dimensions mentioned above. Its purpose is to provide an overview of useful measures but is not in itself a full definition. For the complete treatment of the subject "Software Measurement and Metrics" readers are referred to [1] at the end of this chapter.

Table 4: Attributes and Measures

<i>Dimensions</i>	<i>Attribute</i>	<i>Measures</i>	<i>Issues to consider</i>
COST	Development Effort	Staff effort in hours, days, months or years	Do you include overtime? Which activities are project related and which are overheads?
	Maintenance Effort	Same as above	Do you separate ADAPTIVE, PREVENTIVE and CORRECTIVE maintenance effort?
	Staffing level	Max. Staffing level	How do you count part-time staff members?
		Average Staffing level	How do you count part-time staff members? How do you count people who work full time but only for part of the project?
	Training costs	Cost of training event	Do you include general training or project-specific training?
		Nonproductive staff effort	For each member of staff or all members of staff?
		Cost of Training	Is training a corporate overhead or a project overhead?
		Loss of staff effort during training	As above

<i>Dimensions</i>	<i>Attribute</i>	<i>Measures</i>	<i>Issues to consider</i>
Cost (contd.)		Loss of staff effectiveness (learning curve effects)	How is this measured or assessed?
	Tool costs	Cost of Tools	Is this discounted across all projects that use the tools?
ELAPSED TIME	Duration	Elapsed time in days/months/years	Do you count working time or calendar time? Do you exclude holidays (those in your country, clients' countries or both?)
	Time-scale compression	Difference between estimated time-scale and planned time-scale	How are your estimates produced? What is the accuracy of the measure?
	Scheduled Start Date	Date	What constitutes the formal start of a project?
	Scheduled End Date	Date	What constitutes the formal end of a project: customer acceptance, QA sign off or final payment?
AVAILABILITY, UTILIZATION, PERFORMANCE	Staff availability	Number of staff per staff category required per week	What type of staff do you need, and when do you need them? (staff loading pattern)
	Client availability	Subjective assessment (good, average, bad)	How do you ensure different people interpret this in a uniform way?
	Resource Utilization	Staff effort per week	Is this collected per person or per project?
	Performance	Elapsed time to perform task	Which tasks are measured under what workload conditions?
QUALITY (Customer Perspective)	Usability	User satisfaction survey results	How do you contact your users?
		Number of usability problem reports	Over what timescale do you accumulate this count?
QUALITY (Internal attributes.)	Portability	Percentage of product code/modules using hardware-specific constructs	How do you identify hardware-specific constructs?
	Structure	Linearly independently paths per module (cyclomatic complexity)	How is the measure extracted—manually or using a static analyzer?

<i>Dimensions</i>	<i>Attribute</i>	<i>Measures</i>	<i>Issues to consider</i>
QUALITY (Internal attributes) (contd.)			When is the measure extracted—at the end of software development or after handover to configuration control?
		Number of linearly independent module calling paths	As above
		Maximum depth of module calling tree	As above
COST OF QUALITY (Rework)	Development rework effort	Staff effort	How do you define rework—do you include rework after Inspection or only rework after formal acceptance of a deliverable?
	Post-release rework effort	Staff effort	Do you need to categorize rework into maintenance types such as adaptive, corrective, perfective, preventive?
			How easy is it for your development staff to distinguish different categories of work?
			Do you need to track rework to a particular product release?
QUALITY (Testing related)	Testing defect rate	Defects per week during system testing	How do you ensure that the amount of testing is uniform each week, so that the defect rates are comparable?
	Reliability	Mean time between software failures	What user profile are you using to ensure that results of testing can be used to assess reliability?
			What reliability model are you using to evaluate mean time to failure?
SIZE (Requirements, size Design and Code related)	Requirements	Number of transactions	How do you define a transaction? Are all transactions equal in terms of record size etc.? When do you make the count (of transactions)?

<i>Dimensions</i>	<i>Attribute</i>	<i>Measures</i>	<i>Issues to consider</i>
SIZE (contd.) (Requirements, Design and Code related)		Number and type of requirements	Do you recount them at different stages in the projects?
			Do you include non-functional requirements?
			Do you include constraints? (see Appendix A for the meaning of "constraint")
			Are all requirements equal/ of similar type? When do you make the count? Do you recount at different stages in the projects?
	Specification size	Number of bubbles in Data Flow Diagram or Number of Operations in the Class Diagrams	How do you ensure bubbles/ operations are at the same level of abstraction?
			When do you make the count?
			Do you recount at different stages in the projects?
		Number of entities in the Entity-Relationship Diagrams or the number of classes in the class-association diagrams	When do you make the count?
			Do you recount at different stages in the projects?
		Number of attributes in the Entity Relationship model or number of attributes in the class diagrams	As above
		Number of relationships in the Entity-relationship diagram or number of Associations in the Class Diagrams	As above
Design size	Number of modules in product		How do you define a module?
			When do you make the count?
			Do you recount at different stages in the projects?

<i>Dimensions</i>	<i>Attribute</i>	<i>Measures</i>	<i>Issues to consider</i>
SIZE (contd.) (Requirements, Design and Code related)	Code size	Number of lines of code	At what stages in the construction and testing process do you measure code size?
			What is the convention for counting code size—are comments included? Are compiler directives and declarations included?
		Number of bytes of object code	As above
SIZE (Cost Implications)	Productivity	Lines of code or Number of Function Point per staff month	When do you measure the individual elements?
		Transactions per staff hour	
SIZE (Quality Implications)	Development defect rates	Defects per thousand lines of code per development activity	When do you measure the individual elements?
	Post-release defect rate	Defects per KLOC found by users	Over what time period are the defects counted?
	Test coverage	Percentage of product successfully tested	What measure of the product size will you use? How will you measure coverage?
SIZE (Elapsed time)	Code production rate	Number of lines of code or number of function points produced per week	When do you measure the individual elements?
			Do you measure the actual product of code each week, or the total code produced at the end of the project?
SIZE (Cost and Quality)	Productivity per quality level	Average productivity for products with different quality requirements	What population of products do you use?
			When is the value calculated? Is it re-calculated as new products are completed?
SIZE (Cost and Elapsed Time)	Duration compression productivity impact	Productivity levels for projects under different levels of compression	What population of projects do you use?
			When is the value calculated? Is it re-calculated as new projects finish?
SIZE (Quality and Elapsed Time)	Impact of duration compression on quality	Quality levels for projects under different levels of compression	As above

The attributes and measures shown in Table 4 revolve around a number of specific data collection issues:

- When will the value be collected or calculated?
- If a measure changes over time, will it be reconverted (if possible)?
- How will the value be obtained, manually or by the use of tools?
- Are all the elements that contribute to the value defined? This applies not just to derived measures but also to 'simple' measures such as project effort when you need to be sure that all the activities that contribute to project costs are included and whether or not unpaid overtime is included.
- What methods are used to obtain values that are estimates rather than direct measures?
- If a measure is derived from a set of other measures, is the set of units (from which the raw measures are derived) defined?

To sum up this section, we conclude that it is not very difficult to identify measurable attributes for various dimensions for projects. Factors such as cost and duration are general project management issues that are not specific to software only. However, identifying appropriate measures for *software quality* and *software size* are more difficult.

5.11 Metrics Implementation in Projects

In section 5.8 we discussed about role of metrics in process improvement. You cannot use measures to support process improvement unless you have an on-going measurement program also known as the "metrication program". A quantitative process management means quality planning and defect estimation right at the start of a software project. In this section, we'll discuss a few aspects of this challenge. The next section will add further details on this.

As discussed earlier, quality, along with cost and schedule is a major factor in determining the success of a software project. Table 4 provides details of the measures, attributes and issues in this respect. Let us now discuss how quality activities are planned and managed. Software quality is an important activity for any project.

As we'll see in Chapter 9, the CMM model, an organization at maturity level 4 is expected to have "Quantitative Process Management". Foundations set at Level 4 then help the organization to achieve one of the KPAs at Level 5 called "Defect Prevention" (the vignette in section 5.3.3)

When planning for quantitative management of quality for a project, the key issues are: (1) setting the quality goal and (2) predicting defect levels at intermediate milestones of the projects toward the quality goals set for the project. Defects may exist at any phase of the software life cycle (SDLC). Various defect related measures and metrics were dis-

cussed in section 5.4.

In section 5.2.3 (the vignette for an example) we illustrated how historical performance data from past projects from the process database or process capability baseline are used. In the process capability baseline, defect data are normalized with respect to size or effort. If information normalized with respect to SIZE is to be used for predicting defect levels, then FUNCTION POINT (FP) will be needed as a size estimate for software products. In practice, often the exact data on SIZE is not available (i.e. Exact size of the product in terms of lines of codes or function points/feature points). In such a situation, it is a common practice to use EFFORT estimate for predicting the size by using the "expected productivity level" for the project. The effort estimate can also be directly used by taking advantage of the "defect injection rate" of the software process. Needless to say, this is possible only when past data on projects' performance is available.

As a further example usefulness of the past performance data consider this—suppose that the "Quality Goal" for a project is to have a minimum number of defects in the delivered product. i.e. the quality goal is the expected number of defects during acceptance testing. The quality goal can be "set" to be what is "computed" using the past data. In this case, a standard process is used without modifications and so "standard" results will be expected for quality. Alternatively, a quality goal different from the "standard" can be set. In this case, the process must be modified according to the quality goal. Two primary sources may be used for setting the quality goal: (a) Past data on similar projects and (b) data from process capability baseline.

(a) Past data on similar project

If data on similar projects are used, then the number of defects found during acceptance testing of the current project can be estimated as the product of the number of defects found during the acceptance testing of the similar projects and the ratio of the estimated effort for this project and the total effort of the similar projects. Suppose, the set of similar projects is SP and the current project is P, then the estimate for defects in acceptance testing of P is:

Estimate for Acceptance Testing Defects (P) =

$$[\text{AT Defects (SP)} \times \text{Effort Estimate (P)}] / \text{Actual Effort (SP)}$$

Where AT is Acceptance Testing

This gives the number of defects that can be expected if the same process is used as in the similar projects.

(b) Use of Capability Baseline

If the data from Process Capability Baseline are used, then several methods can be used to compute this value. Table 5 shows some examples of values in process capability baseline for defect estimation.

Table 5: Parameters for Defect Estimation

Productivity	4-18 FP/person-month (Avg.: 10 FP/person month)
Delivered Defects	0.04-0.09 Defects/FP (Avg.: 0.06 Defects/FP)
In Process DRE	90%-95% (Avg.:93%)
Defect Injection Rate	0.8-1.2 Defects/FP (Avg.: 0.95 Defects/FP)

If the quality target is set as the number of defects per Function Point, then SIZE in Function Points is estimated by using appropriate method for counting the Function Point (beyond the scope of this chapter—see www.ifpug.org for details of Function Points). In this case, the “expected number of defects” is the product of the quality figure and estimated size. The following sequence of steps is used:

1. Set the Quality Goal in terms of Defects/Product Size (From the Process Capability Baseline)
2. Estimate the “expected productivity level” for the project
3. Estimate the Product Size (Expected Productivity x Estimated Effort)
4. Estimate the Number of Acceptance Test Defects as : Quality Goal x Estimated Size

Sometimes, it is more useful to set the quality goal in terms of the “Defect Removal Efficiency” of the software process (This term was discussed in section 5.4). In this situation, the number of defects to be expected during acceptance testing can be determined from the “defect injection rate”, the target-in-process removal efficiency, and the estimated size. The following sequence of steps is used:

1. Set the Quality Goal in terms of Defect Removal Efficiency (DRE)
2. Estimate the Product Size (as discussed earlier)
3. Estimate the total number of defects (Defect Injection Rate x Estimated Size)
4. Estimate the number of Acceptance Testing Defects from the total number of defects and Quality Goal

Table 6 below shows example of typical quality goals in a software project.

Table 6: Quality Goals for a Project (An Example)

Goals	Value	Basis for setting goal	Prevailing Norms
Total number of defects injected	145	0.033 defects per person-hour (This is 10% better than Project X)	0.052 defects per person-hour
Quality (acceptances defects)	3	3% or less of total estimated number of defects (defects reported by customer after product delivery)	5% of estimated number of defects
Productivity (FP per person-month) (It should be noted that productivity may vary depending on the development language used)	55	3.5% productivity improvement over Project X	50
Schedule	Delivery on Time (0% schedule slippage)		Slippage within plus or minus 10%

5.12 Benefits of Measurement and Metrics for Project Tracking and Control

The basic purpose of measurement in a project is to be able to establish and effective project control and project tracking mechanism. In this section, we discuss some concepts related to metrics and measurement and the basic metrics useful for controlling a software project. Statistical Process Control (SPC) is among the well-known approaches for effective project control; however, a discussion on that is not in the scope of this chapter. (Readers can refer Appendix C for this)

In the light of the discussion so far, we can appreciate how software metrics can be used to quantitatively characterize various quality aspects. *Process metrics* quantify attributes of the software process or the development environment, whereas *product metrics* are measures for the software products (The topic of *process quality* and *product quality* has been dealt with in Chapter 4). Product metrics remain independent of the process used to produce the product. Examples of *process metrics* include *productivity*, *quality*, *resource metrics*, *training metrics*, *defect injection rates* and *defect removal efficiency* (these were discussed in the previous sections). Examples of *product metrics* include *size*, *reliability*, *quality* (quality can be viewed as a product metric as well as a process metric), *code complexity*, and *functionality*.

The use of metrics for project tracking and control necessitates that measurement be made to obtain data. This, in turn, means that for a metrics program to be successful, one must clearly understand the goals for collecting data as the models to be used for making judgments based on the data. In general, metrics to be used and measurements to be

made will depend on the project goals and the goals of the organization as a whole.

Schedule is one the most important metrics because most fixed cost projects are driven by schedules and deadlines. It is easy to measure because calendar time is usually used. *Effort* is the main resource consumed in a software project. Tracking project efforts is thus a key project management activity. For this, the technique of "earned value analysis" is very useful (it is discussed in brief in the next section). It helps to determine if the project is executing within budget. Such data helps in making statements (for example) like—"The cost of current Project Y is likely to be about 40% more than an earlier project X" which was executed with very similar parameters or "The project is likely to finish within budget".

Now let discuss how measurements and metrics help a project manager make some predictions based on the past performance data. This is because defects have a direct relationship to software quality, tracking of defects, which, in turn, critical for ensuring quality. Table 7 shows an example of a hypothetical project estimates on defects in various project phases.

A large software project may include thousands of defects that are

Table 7: Defect Estimates (An Example)

Project Phase	Estimates Number of Defects (to be detected)	% of Defects to be detected	Basis for estimation
Requirements and Design review	25	20%	Similar Project in the past and process capability baseline
Code Review	25	20%	Similar Project in the past and process capability baseline
Unit Testing	60	40%	Similar Project in the past and process capability baseline
Integration and Regression Testing	30	15%	Similar Project in the past and process capability baseline
Acceptance Testing	5	5%	Similar Project in the past and process capability baseline
Total estimated number of defects to be detected	145	100%	

found by different people at different stages. In practice, it may happen that the person who fixed the defect may not the same person who found and reported the defect. Generally, the project manager would want to remove most of the defects found before the final delivery of the software

product. To do this, formal mechanisms become essential for the defect tracking and reporting process. This means that defects must be logged and their closure must be tracked.

However, mere logging defects and tracking them is not sufficient to support other desirable analysis to help the project manager. The project manager needs to understand what percentage of defects are caught where, phases in which defects are detected. In order to understand the defect removal efficiency of various quality control tasks (such as reviews and testing) and thereby improve their performance, not only one must know where a defect is detected but also where it was injected (the discussion in section 5.4.4). Thus, for each defect, it is essential to provide information about the phase in which the defect was introduced.

As noted earlier, Size is another fundamental metric because many data are normalized (defect density, productivity etc) are normalized with respect to size. Without the data on size, it is very difficult to do performance prediction using the past data. Function Points/Feature Points provide the uniformity because productivity measured in lines of code, differs with programming language.

To sum up, we note that a good project planning practice requires a plan to monitor the progress of the project. Tracking the progress, in turn, requires measurements and associated goals. Having established goals, measures and suitable metrics, it becomes possible to assess project's performance against the goals. The next section shows how the "earned value analysis" technique can be used to proactively track the project

5.13 Earned Value Analysis

5.13.1 Basics of EVA

Earned value analysis (EVA) in various forms is the most commonly used method of *performance measurement*. It integrates SCOPE, COST (or RESOURCE) and SCHEDULE measures to help project teams to assess project performance. The most typical use of EVA is seen in fixed cost projects. EVA is related to EVM—in the sense that Earned Value Management (EVM) is a methodology used to measure and communicate the real physical progress of a project taking into account the work complete, the time taken and the costs incurred to complete that work. *Earned Value* helps evaluate and control project risk by measuring project progress in monetary terms.

Earned value (EV) involves calculating *three key values* for each project activity:

1. *The Planned Value (PV)*: also known as the *budgeted cost of work sched-*

ule (BCWS), is that portion of the approved cost estimate planned to be spent on the project activity during a given period

2. *The Actual Cost (AC)*: also known as the *actual cost of work performed (ACWP)*, is the total of costs incurred in accomplishing work on the activity during a given period. This Actual Cost must correspond to whatever was budgeted for the PV and EV (example: direct efforts in hours, direct costs, or all the costs including indirect costs)
3. *The Earned Value (EV)*: also known as the budgeted cost of work performed (BCWP) is the value of the work actually completed.

These three key values are used in combination to provide measures of whether or not work being done is being accomplished as planned. This way, it enables proactive project tracking. An example of EVA is provided later in this section. Having understood the three key values of EVA, let us now see how these EVA based measurement and metrics help projects to take corrective actions.

5.13.2 Use of the three key EVA-based measures for Project Tracking

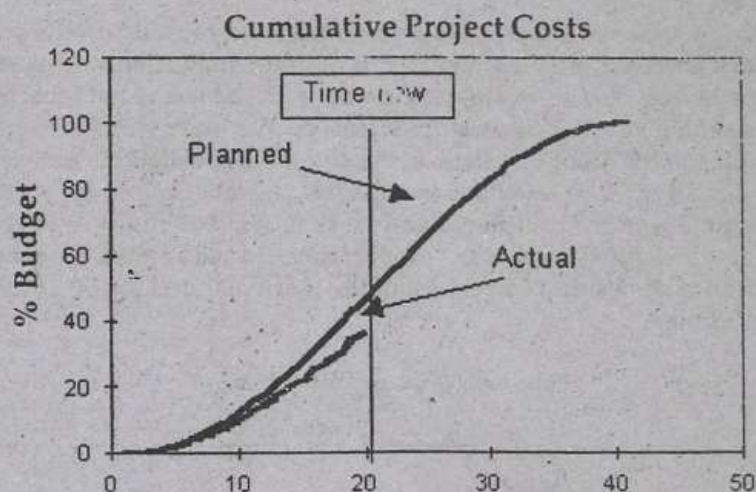
The three key values are used in combination to provide measures of whether the project work is being accomplished as planned (this is the important aspect of metrics-based proactive project tracking). The most commonly used measures are the *Cost Variance (CV)* and the *Schedule Variance (SV)*. These two values can be converted to efficiency indicators to reflect the cost-and schedule performance of a project. The *cost performance index* is the most commonly used *cost-efficiency indicator*. Here is how the calculations are done:

Cost Variance (CV)	= EV minus AC or BCWP minus ACWP
Schedule Variance (SV)	= EV minus PV or BCWP minus BCWS
Cost Performance Index (CPI)	= EV divided by AC or BCWP/ACWP
Schedule Performance Index (SPI)	= EV divided by PV or BCWP/BCWS

The basics of Earned Value can best be shown on the ubiquitous 'S-Curve'. The S-curve in its simplest form is a graph showing how project budget is planned to be spent over time. The BCWS (or Planned Value—PV) curve is derived from the *Work Breakdown Structure*, the project budget and the Project Master Schedule. The cost of each Work Package is calculated and the cumulative cost of completed Works Packages is shown based on the planned completion dates shown in the Master Schedule. The ACWP (or the Actual Cost—AC) curve is found by actual measurement of the work completed. Actual costs recorded from invoices and project team member's time sheets. This appears a daunting task but it can be very simple with sufficient planning and organizing. The BCWP (or the Earned Value -EV) is calculated from the measured work complete and the budgeted costs for that work.

Earned Value = (Percentage project complete) X (Project Budget). Sec-

tion 5.14.4 shows how conclusions can be drawn from the values calculated as above.



5.13.3 EVA Benefits to Project Management

The benefits to project management of the earned value approach come from the disciplined planning conducted and the availability of metrics which show real variances from plan in order to generate necessary corrective actions. However, the success of deploying EVA techniques depends of certain things (such as clarity on requirements and scope, good configuration management practices etc.). This is discussed below.

Earned value requires that the project be fully defined at the outset and then a bottom-up plan be created. This allows *measurement* to take place during the entire period of performance, from 1 percent to 100 percent of the project's lifecycle. The power in Earned Value Analysis technique is that it provides accurate and reliable readings of performance from as early as 15 percent into the project. Project manager can use these performance readings to predict how much it will cost to complete the project within a narrow band of values. If these early warning signals convey unacceptable readings to the project manager, steps can be immediately taken to avoid the undesired results. In the next section, we show how conclusions can be drawn on the basis of calculated values using the EVA technique.

5.13.4 EVA: An Illustrative Example

Baseline—The baseline plan in the table below shows that 6 work units (A-F) would be completed at a cost of Rs.100 for the period covered by a hypothetical report for the purpose of this illustration.

Baseline Plan Work Units

	A	B	C	D	E	F	Total
Planned value (Rs.)	10	15	10	25	20	20	100

Schedule variance—As work is performed, it is "earned" on the same basis as it was planned, in money terms or other quantifiable units such as labor hours. Planned value compared with earned value measures the monetary volume of work planned vs. the equivalent monetary volume of work accomplished. Any difference is called a "schedule variance". In contrast to what was planned, the table below shows that work unit D was not completed and work unit F was never started, or Rs. 35 of the planned work was not accomplished. As a result, the schedule variance shows that 35 percent of the work planned for this period was not done.

Schedule Variance Work Units

	A	B	C	D	E	F	Total
Planned value (Rs.)	10	15	10	25	20	20	100
Earned value (Rs.)	10	15	10	10	20	-	65
Schedule variance	0	0	0	-15	0	-20	-35 = -35%

Cost variance—Earned value compared with the actual cost incurred (with information received from a formal project accounting systems) for the work performed provides an objective measure of planned and actual cost. Any difference is called a cost variance. A negative variance means more money was spent for the work accomplished than was planned. The table below shows the calculation of cost variance. The work performed was planned to cost Rs. 65 but its actual cost turns out to be Rs.91. The cost variance is 40 percent.

Cost Variance Work Units

	A	B	C	D	E	F	Total
Earned value (Rs.)	10	15	10	10	20	-	65
Actual cost (Rs.)	9	22	8	30	22	-	91
Cost variance	1	-7	2	-20	-2	0	-26 = -40%

Spend comparison—The typical spend comparison approach, whereby the party executing the contractual work reports actual expenditures against planned expenditures is not related to the work that was accomplished. The table on the next page shows a simple comparison of planned and actual spending, which is unrelated to work performed and therefore not a useful comparison. The fact that the total amount spent was Rs.9 less than planned for this period is not useful without the comparisons with work accomplished.

Spend Comparison Approach Work Units

	A	B	C	D	E	F	Total
Planned spend (Rs.)	10	15	10	25	20	20	100
Actual spend (Rs.)	9	22	8	30	22	-	91
Variance	1	-7	2	-5	-2	20	9 = 9%

Use of Earned Value Data—The benefits to project management of the earned value approach come from the disciplined planning conducted and the availability of metrics which show real variances from plan in order to generate necessary corrective actions.

Conclusions using Earned Value Analysis—*Schedule Performance Index* is a ratio of *Earned Value* and the *Planned value* of completed works. An $SPI < 1$ is not good. Also, *Cost Performance Index* is a ratio of *Earned Value* and the *Actual costs* of completed works. A $CPI < 1$ is not good. A lot more analysis is possible using the three key values of the EVA techniques. Interested readers may like to refer to any standard book on the subject of Project Management.

5.13.5 Essentials for EVA Success

The example above shows that the EVA technique as a tool for proactive project tracking, however appealing and powerful, is not easy to implement. It requires a considerable amount of project management discipline (such as good change control practices, project accounting control etc.). In that sense, one can say that there are "Ten Commandments" to implement Earned Value in Projects! These are summarized below:

1. Define Work Scope
2. Create an Integrated Bottom-Up Plan and Establish detailed measurement cells (called Project Accounting Systems—PAS for various aspects of projects).
3. Formally Schedule PAS
4. Assign Each PAS to an Executive for Performance
5. Establish a Baseline that Summarizes various PAS
6. Measure Performance Against Schedule
7. Measure Cost Efficiency Against the Costs Incurred
8. Forecast Final Costs Based on Performance
9. Manage Remaining Work
10. Manage Baseline Changes

5.14 Planning for Metrics Program

Like any other software or business activity, the introduction of a metrication program must be properly planned. You will need to consider the type of metrication program intended for your organization:

- One-off: this is a metrication program that has a well-defined goal, which can be addressed by performing a single data collection and analysis activity. This type of program is typical of an evaluation exercise (either a formal experiment or a case study). The important point is that once the goal has been achieved, the metrication program ends)
 - On-going: this is a metrication program that is assumed to continue indefinitely. Metrication program of this type have goals related to issues such as "supporting project management and quality control as well as the SEI-CMM level achieved.
 - Mixed: this is a metrication program that has some one-off goals and some ongoing ones. The distinction between the two types of goals must be made clear in the metrication plans. Your plans must also consider the stages in the metrication program to be deployed at your organization. These are somewhat similar to a software life cycle:
1. Goal setting
 - Definition of goals
 - Validating the goals
 2. Data collection system specification and design
 - Formally defining all the measures
 - Defining the processes needed to extract, validate, store and analyze measures
 3. Installation
 - Providing training to all concerned
 - Installing any required extraction, storage and analysis tools
 4. Operation
 - Data validation—you must ensure that you have procedures for rejecting submitted data and handling re-submitted data and with valid reasons to defend rejections
 - Extracting, analyzing and reporting on data
 - Providing on-going support for software engineering staff involved with data collection system
 5. Maintenance
 - Amending the data collection system if there are deficiencies
 - Enhancing the data collection system as development processes and data requirements change
 6. Retirement i.e. closedown of the data collection system

5.15 Issues in Software Measurements & Metrics Program Implementation

Metrication Program in an organization is unlikely to succeed unless the software development staff who provide the raw data are committed to the cause of metrics. In practice, there are very good reasons

why software development staff perceive metrication program as an unwelcome overhead and sometimes even as a major threat! Metrication program require software engineers to record information about their development activities. The information required usually includes data indicating both their own productivity and the quality of products they produce. If you introduce a system, which asks development staff to keep records of what they view as their personal limitations (e.g. the defects/faults which they introduced into their code, or the time takes them to design and code), then, it is not surprising to find that many staff are not highly motivated to keep accurate records. In addition to this, the usual project pressures take their toll on these good record-keeping habits.

Over and above this, if your staff do not know why the data is being collected or how it is going to be analyzed and used, then you are very likely to be faced with a large amount of incorrect and/or missing data. To minimize mis-recording, you must be committed enough to the data collection activity to take the time and effort needed to motivate your development staff to keep accurate records. Strategies for motivation include:

- Consultation and involvement
- User-friendly data collection systems/tools
- Awareness creation through training
- Feedback sessions
- Technical leadership
- Top Management support

5.15.1 Consultation and Involvement (Selling the Metrics Program)

If you want to introduce a metrication program in the organization, you will need to organize some consultation meetings. Consultation meetings should explain by the metrication program is needed, what it is expected to deliver, give some introduction to metrics and provide the opportunity for development staff to contribute to the program.

Managers should tell staff why they need the data they wish to see collected. It is even more important that staff and managers agree how the data is going to be used. If managers agree that data will not be used for personal appraisal, they must keep to that agreement whatever the temptations; otherwise the whole basis for obtaining valid data will be compromised.

It is usual to see concerns raised in these meeting about who "own" the data. This is a difficult issue to resolve. On the one hand, if you want to obtain commitment to the metrication program, development staff and/or their line managers must feel a degree of "ownership" toward the program. On the other hand, you cannot obtain the full benefit of a measurement program unless data from different projects can be

collected and analyzed to provide information about the organization as a whole. Once the data are delivered from a project to another part of the organization, the originators of the data cannot realistically be allowed to assume that they still "own" all the data.

Unless you are not intending to collate data from different projects, it is better to make clear that the "company owns the data" (data as corporate asset), in the same way that the company owns the software produced by the development staff. Individual project staff and their direct line managers should be at liberty to store, analyze and use the data they generate. In addition, they should be at liberty to collect and use data that is not needed to support corporate goals. Nonetheless, when data values are delivered to another part of the organization, they must be owned by the people who receive them.

5.15.2 User-Friendly Data Collection Systems and Tools

Every effort must be made to make data collection as easy as possible. Methods of minimizing overheads include automation and incorporating data collection into working practices of the projects. Efforts to make the system easier to use will usually reduce the incidence of deliberate mis-recording, ("cooking the metrics" for illicit purpose as it is called) as well as the amount of unintentional mis-recording. (Discussion of issues relating to automation is beyond the scope of this chapter.)

For manually collected data, in particular, the data, which depends on subjective assessment, it is vital both that data providers use well-understood definitions and that the amount of data to be collected is minimized. Information about changes resulting from defects/faults or change requests is important if you are interested in "quality of metrics" and it is important to have an appropriate classification system. The particular categories will depend on the particular development processes used in a particular environment and on the way product support is organized.

5.15.3 Training

For any non-trivial data collection process training is needed. It is the best tool for awareness creation. This should cover training in both the automated and the manual parts of the metrics system. In addition, software staff should benefit from training in software data analysis.

5.15.4 Feedback

Most metrication programs provide a set of standard reports to the management, but it is important to provide some feedback to the individuals who provide the data as well. You should report the results of data collection exercise back to staff quickly and regularly. Project staff

will be de-motivated if they feel that the data they collected is not being used constructively. Feedback plays an even more important role if the metrication program is itself on trial (as most programs are for the first few years). One typically used approach is to identify "metrics champions" early on, work closely with them on designing metrics suitable to their projects, win their confidence and later use these champions to act as metrics advocates. Such project managers can be encouraged to spread success stories by word of mouth, by testimonials at carefully designed and planned workshops and by publishing articles.

5.15.5 Technical Issues

Technical issues to grapple with are: "operational definition of LOC (lines of Code) i.e. what basis should be used to count the number of lines of code (for example, should it include comments lines or only executable lines? Should it include compiler directives?). Extreme caution needs to be exercised when computing defect rates of two products if the operational definition of LOC, defects and time frame are not identical.

Also there are several methods to use for counting the function points and organizations need to make a careful choice by working with experts in the field. An excellent reference for a detailed discussion on this is [1].

5.15.6 Top Management Commitment

As said in first few chapters, like Quality Management System (QMS), metrics programs will fail without the whole-hearted support of top management. However, like quality assurance, and project management, software metrics is a "risk reduction methodology", not directly, a "cost reduction technology". This means that it may be as difficult to sell software metrics to senior management, as it is to motivate staff to collect data. In general, the following points can be made:

- Software staff already spend up-to 10 percent of their time generating data by filling in time sheets, using configuration control systems, filling in inspection reports and completing test reports. Most data collection systems do not significantly increase this burden (and good systems decrease it). Furthermore, unless this data is stored and analyzed, it is wasted.
- The largest cost associated with metrics program is the effort involved in data storage and data analysis. This can run at about 5% of project costs. However, if you are intending to obtain ISO 9001 certification, you are required to store and analyze quality records anyway, so the costs would be part of your normal QMS costs.

5.16 Object Oriented Metrics: An Overview

Object-oriented design and development is becoming very popular in today's software development environment. Object oriented development requires not only a different approach to design and implementation, it requires a different approach to software metrics. Since object oriented technology uses objects and not algorithms as its fundamental building blocks, the approach to software metrics for object-oriented programs must be different from the standard metrics set discussed so far. Some metrics, such as lines of code and *cyclomatic complexity*, have become accepted as "standard" for traditional functional/ procedural programs, but for object-oriented, there are many proposed object oriented metrics in the literature as will be seen in the next two sections. The question is, "Which object oriented metrics should a project use, and can any of the traditional metrics be adapted to the object oriented environment?" Extensive research has been done in the area of Object Oriented Metrics. A complete discussion on the topic is beyond the scope of this chapter. In the reference section some good links have been provided to the topic of OO metrics.

Object-Oriented Analysis and Design of software provide many benefits such as reusability, decomposition of problem into easily understood object and the aiding of future modifications. But the OOAD software development life cycle is not easier than the typical procedural approach. Therefore, it is necessary to provide dependable guidelines that one may follow to help ensure good OO programming practices and write reliable code. Object-Oriented programming metrics is an aspect to be considered. OO-Metrics is to be a set of standards against which one can measure the effectiveness of Object-Oriented Analysis techniques in the design of a system. As said above, the question is, "Which object oriented metrics should a project use, and can any of the traditional metrics be adapted to the object oriented environment?" In the sections that follow, we try to provide an answer to these questions.

5.16.1 Object Oriented Metrics: Details

In this section we discuss some OO metrics, which can be applied to analyze source code as an indicator of quality attributes. The source code could be any OO language.

5.16.1.1 Metrics for OO Software Development Environments

[1] Methods per Class

Average number of methods per object class = $\frac{\text{Total number of methods}}{\text{Total number of object classes}}$

- A larger number of methods per object class complicates testing due to the increased object size and complexity
- if the number of methods per object class gets too large extensibility will be hard
- A large number of methods per object class may be desirable because subclasses tend to inherit a larger number of methods from super-classes and this increases code reuse.

[2] Inheritance Dependencies

Inheritance tree depth = max (inheritance tree path length)

- Inheritance tree depth is likely to be more favorable than breadth in terms of reusability via inheritance. Deeper inheritance trees would seem to promote greater method sharing than would broad trees
- A deep inheritance tree may be more difficult to test than a broad one
- Comprehensibility may be diminished with a large number inheritance layers

[3] Degree of Coupling Between Objects

The topic "Principles of Coupling and Cohesion" is discussed in Appendix C.

Average number of uses dependencies per object = total number of arcs / total number of objects
 $\text{arcs} = \max (\text{number of uses arcs}):$ in an object <<uses>> network arcs—attached to any single object in a <<uses>> network

- A higher degree of coupling between objects complicates application maintenance because object interconnections and interactions are more complex.
- The higher the degree of uncoupled object the more objects will be suitable for reuse within the same applications and within other applications.
- Uncoupled objects should be easier to augment than those with a high degree of 'uses' dependencies, due to the lower degree of interaction.
- Testability is likely to degrade with a more highly coupled system of objects.
- Object interaction complexity associated with coupling can lead to increased error generation during development.

[4] Degree of Cohesion of Objects

Degree of Cohesion of Objects = Total Fan-in for All Objects / Total Number of Objects (The term "Fan-in" has been discussed on page 291).

- Low cohesion is likely to produce a higher degree of errors in the development process. Low cohesion adds complexity which can translate into a reduction in application reliability.
- Objects which are less dependent on other objects for data are likely to be more reusable.

[5] Object Library Effectiveness

Average number = $\text{Total Number of Object Reuses} / \text{Total Number of Library Objects}$

- If objects are actually being designed to be reusable beyond a single application, then the effects should appear in object library usage statistics.

[6] Factoring Effectiveness

Factoring Effectiveness = $\text{Number of Unique Methods} / \text{Total Number of Methods}$

- Highly factored applications are more reliable for reasons similar to those, which argue that such applications are more maintainable. The smaller the number of implementation locations for the average task, the less likely that errors were made during coding
- The more highly factored an inheritance hierarchy is the greater degree to which method reuse occurs
- The more highly factored an application is, the smaller the number of implementation locations for the average method

[7] Degree of Reuse of Inheritance Methods

Percent of Potential Method Uses Actually Reused (PP):

$PP = (\text{Total Number of Actual Method Uses} / \text{Total Number of Potential Method Uses}) \times 100$

Percent of Potential Method Uses Overridden (PM):

$PM = (\text{Total Number of Methods overridden} / \text{Total Number of Potential Method Uses}) \times 100$

- Defining methods in such a way that they can be reused via inheritance does not guarantee that those methods are actually reused.

[8] Average Method Complexity

Average method complexity = $\text{Sum of the cyclomatic complexity of all Methods} / \text{Total number of application methods}$

- More complex methods are likely to be more difficult to maintain.
- Greater method complexity is likely to lead to a lower degree of overall application comprehensibility.

- Greater method complexity is likely to adversely affect application reliability.
- More complex methods are likely to be more difficult to test.

[9] Application Granularity

Application granularity = total number of objects / total function points

- One of the goals of object-oriented design is finer granularity. The purpose is to achieve a greater level of abstraction than possible with data/procedures-oriented design.
- An application constructed with more finely granular objects (i.e. a lower number of functions per object) is likely to be more easily maintained because objects should be smaller and less complex.
- More finely granular objects should also be more reusable. ... Therefore, each object's behavior should be more easily understood and analyzed.

5.16.1.2 Chidamber & Kemerer's Metrics Suite

Chidamber and Kemerer's metrics suite for OO Design is the deepest research in OO metrics investigation. They have defined six metrics for the OO design.

[1] Weighted Methods per Class (WMC)

It is defined as the sum of the complexities of all methods of a class.

- The number of methods and the complexity of methods involved is a predictor of how much time and effort is required to develop and maintain the class.
- The larger the number of methods in a class the greater the potential impact on children, since children will inherit all the methods defined in the class.
- Classes with large numbers of methods are likely to be more application specific, limiting the possibility of reuse.

[2] Depth of Inheritance Tree (DIT)

It is defined as the maximum length from the node to the root of the tree.

- The deeper a class is in the hierarchy, the greater the number of methods it is likely to inherit, making it more complex to predict its behavior.
- Deeper trees constitute greater design complexity, since more methods and classes are involved.

- The deeper a particular class is in the hierarchy, the greater the potential reuse of inherited methods.

[3] *Number of Children (NOC)*

It is defined as the number of immediate subclasses.

- The greater the number of children, the greater the reuse, since inheritance is a form of reuse.
- The greater the number of children, the greater the likelihood of improper abstraction of the parent class. If a class has a large number of children, it may be a case of misuse of sub-classing.
- The number of children gives an idea of the potential influence a class has on the design. If a class has a large number of children, it may require more testing of the methods in that class.

[4] *Coupling between Object Classes (CBO)*

It is defined as the count of the classes to which this class is coupled. Coupling is defined as: Two classes are coupled when methods declared in one class use methods or instance variables of the other class.

- Excessive coupling between object classes is detrimental to modular design and prevents reuse. The more independent a class is, the easier it is to reuse it in another application.
- In order to improve modularity and promote encapsulation, inter-object class couples should be kept to a minimum. The larger the number of couples, the higher the sensitivity to changes in other parts of the design, and therefore maintenance is more difficult.
- A measure of coupling is useful to determine how complex the testing of various parts of a design are likely to be. The higher the inter-object class coupling, the more rigorous the testing needs to be.

[5] *Response For a Class (RFC)*

It is defined as number of methods in the set of all methods that can be invoked in response to a message sent to an object of a class.

- If a large number of methods can be invoked in response to a message, the testing and debugging of the class becomes more complicated since it requires a greater level of understanding on the part of the tester.
- The larger the number of methods that can be invoked from a class, the greater the complexity of the class.
- A worst-case value for possible responses will assist in appropriate allocation of testing time.

[6] Lack of Cohesion in Methods (LCOM)

It is defined as the number of different methods within a class that reference a given instance variable.

- Cohesiveness of methods within a class is desirable, since it promotes encapsulation.
- Lack of cohesion implies classes should probably be split into two or more subclasses.
- Any measure of disparateness of methods helps identify flaws in the design of classes.
- Low cohesion increases complexity, thereby increasing the likelihood of errors during the development process.

5.16.1.3 MOOD (Metrics for Object Oriented Design)

The MOOD metrics set refers to a basic structural mechanism of the OO paradigm as *encapsulation* (MHF and AHF), *inheritance* (MIF and AIF), *polymorphism* (PF), *message-passing* (CF) and are expressed as quotients. The set includes the following metrics:

[1] Method Hiding Factor (MHF)

MHF is defined as the ratio of the sum of the invisibilities of all methods defined in all classes to the total number of methods defined in the system under consideration.

The invisibility of a method is the percentage of the total classes from which method is not visible.

Note: inherited methods not considered

[2] Attribute Hiding Factor (AHF)

AHF is defined as the ratio of the sum of the invisibilities of all attributes defined in all classes to the total number of attributes defined in the system under consideration.

[3] Method Inheritance Factor (MIF)

MIF is defined as the ratio of the sum of the inherited methods in all classes of the system under consideration to the total number of available methods (locally defined plus inherited) for all classes.

[4] Attribute Inheritance Factor (AIF)

AIF is defined as the ratio of the sum of inherited attributes in all classes of the system under consideration to the total number of available attributes (locally defined plus inherited) for all classes.

[5] Polymorphism Factor (PF)

PF is defined as the ratio of the actual number of possible different polymorphic situation for class C_i to the maximum number of possible distinct polymorphic situations for class C_i .

[6] Coupling Factor (CF)

CF is defined as the ratio of the maximum possible number of couplings in the system to the actual number of couplings not imputable to inheritance.

5.16.2 Adapting Traditional Metrics to the Object Oriented Environment

Research shows that there is considerable disagreement in the field about software quality metrics for object oriented systems. Some researchers and practitioners contend traditional metrics are inappropriate for object oriented systems. There are valid reasons for applying traditional metrics, however, if it can be done. The traditional metrics have been widely used, they are well understood by researchers and practitioners, and their relationships to software quality attributes have been validated.

Table 8 presents an overview of the metrics for object oriented systems. Traditional metrics can be applied within the structures and confines of object oriented systems. The first three metrics in Table 8 are examples of traditional metrics applied to the object oriented structure of methods instead of functions or procedures. The next six metrics are specifically for object oriented systems and the object oriented construct applicable is indicated.

Table 8: Overview of Metrics for Object Oriented Systems

Source	Metric	Object-oriented construct
Traditional	Cyclomatic complexity (CC)	Method
Traditional	Lines of Code (LOC)	Method
Traditional	Comment percentage (CP)	Method
NEW Object-Oriented	Weighted methods per class (WMC)	Class/Method
NEW Object-Oriented	Response for a class (RFC)	Class/Message
NEW Object-Oriented	Lack of cohesion of methods (LCOM)	Class/Cohesion
NEW Object-Oriented	Coupling between objects (CBO)	Coupling
NEW Object-Oriented	Depth of inheritance tree (DIT)	Inheritance
NEW Object-Oriented	Number of children (NOC)	Inheritance

Finally, we end this section with a few concluding remarks. Researchers have found that a combination of "traditional" metrics and

metrics that measure structures unique to object oriented development is most effective. This allows developers to continue to apply metrics that they are familiar with, such as complexity and lines of code to a new development environment. However, now that new concepts and structures are being applied, such inheritance, coupling, cohesion, methods and classes, metrics are needed to evaluate the effectiveness of their application. Metrics such as Weighted Methods per Class, Response for a Class, and Lack of Cohesion are applied to these areas. The application of a hierarchical structure also needs to be evaluated through metrics such as Depth in Tree and Number of Children.

Conclusion

Software metrics are numerical data related to software development. There are various ways to classify metrics. Software metrics is all about measurement, which, in turn, involves numbers; the use of numbers to improve the process of developing software. Software Metrics are applicable to the whole development lifecycle. Metrics collected from work product review help software engineers to spot error-prone components before they get as far as coding.

Measurement and Metrics can also be used to control a project as it progresses. Techniques like Earned Value Analysis provide visibility to management by generating early signals. This helps to adopt a proactive approach to project management by handling trouble areas before they become too big to manage. Thus, measurements and metrics play a significant role in effective project management. They relate to the four functions of management as follows:

1. **Planning**—Metrics serve as a basis of cost estimating, training planning, resource planning, scheduling, and budgeting.
2. **Organizing**—Size and schedule metrics influence a project's organization.
3. **Controlling**—Metrics are used to status and track software development activities for compliance to plans.
4. **Improving**—Metrics are used as a tool for process improvement and to identify where improvement efforts should be concentrated and measure the effects of process improvement efforts.

Although their usage is not as widespread as the traditional metrics, there are a number of object oriented metrics which provide valuable information to object oriented developers and project managers.

REFERENCES

- [1] *Applied Software Measurement*—Capers Jones (McGraw Hill Computing Series)
- [2] *Software Metrics : SEI Curriculum Module SEI-CM-12-1.1* December 1988
- [3] *Software Metrics : A Practitioner's Guide to Improved Product Development*—Møller and Paulish (Chapman & Hall)

- [4] *Measuring the Software Process*—William A. Florac and Anita D. Carleton (Addison-Wesley)
- [5] *The Software Measurement Guidebook*—Software Productivity Consortium (International Thomson Computer Press)
- [6] IFPUG (International Function Point Users' Group)—www.ifpug.org/
- [7] *Software Engineering: A Practitioner's Approach*—Roger S. Pressman (McGraw Hill International Edition Computer Science Series)
- [8] An up-to-date list of World Wide Web references that are relevant to technical metrics can be found at the web-site <http://www.mhhe.com/engcs/compsci/pressman/resources/tech-metrics.mhtml>
- [9] *Object-Oriented Design Measurement*—Scott A. Whitmire (John Wiley & Sons Inc)
- [10] *Object-Oriented Metrics: Measures of Complexity* by Brian Henderson-Sellers December 4, 1995; Prentice Hall; ISBN: 0132398729
- [11] *A Metrics Suite for Object Oriented Design* This paper proposes six OO design metrics that the authors (Chidamber, S.R., Kemerer, C.F.) claim are based firmly in theory of measurement and ontology of objects. Traditional metrics do not take OO notions into account.
- [12] *A Validation of Object-Oriented Design Metrics as Quality Indicators* This paper presents the results of a study conducted at the University of Maryland in which the authors experimentally investigated the suite of Object-Oriented (OO) design metrics introduced by Chidamber & Kemerer.
- [13] *Software Metrics : A Guide to Planning, Analysis and Application*—C.R. Pandian, Quality Improvement Consultants, Hyderabad (qic@rediffmail.com)
- [14] *Software Requirements and Estimation*—Swapna Kishore and Rajesh Naik (Tata McGraw Hill)
- [15] *Managing the Software Process*—Watts S. Humphrey (Addison-Wesley Publishing Company)

WALKTHROUGHS AND INSPECTIONS

6.1 Overview

In this chapter we are going to discuss about the various review techniques called WALKTHROUGHS, DESK CHECKS, and INPSECTIONS. These are, "static" testing techniques. We'll discuss the, why and how of these techniques and will also present the viewpoints of various experts known to be associated with these techniques. We'll also discuss relative benefits and drawbacks of these techniques. The objective is to explain the contribution of these techniques towards ensuring the quality of product to be delivered to the customer.

6.2 Introduction

6.2.1 Why Reviews at All?

Technical work needs reviewing for the same reason that pencils need erasers: To err is human! The second reason why we need technical reviews is that although people are good at catching some of their own errors, large classes of errors escape the originator more easily than they escape anyone else. A review is a way of using the diversity and power of a group of people to:

1. Point out needed improvements in a product of a single person or team;
2. Confirm those parts of a product in which improvement is either not desired or not needed;
3. Achieve technical work of more uniform, or at least more predictable quality than can be achieved without reviews, in order to make technical work more manageable

A formal technical review (FTR) is a verification activity that is performed by software engineers. Ref [2] In a general sense of the term,

a "review" is performed on any "work product" (such as a project plan, a piece of code etc). A work product is something that expends efforts and has a role in the execution of a software project. The FTR is actually a class of reviews that include *walkthroughs*, *inspections*, and *round-robin reviews* and other small group technical assessments of software. Ref [1] Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended by those for whom it is intended. In this chapter we'll discuss the various types of reviews, degree of formality involved in reviews, the various roles involved in reviews and what goes in making reviews and inspections successful as well as some typical pitfalls involved.

Another perspective on "reviews and inspection" is in terms of control. A control is anything that tends to cause the reduction of risk. Controls accomplish this either by reducing harmful effects or by reducing the frequency of occurrence. Whether Reviews and Inspections and the similar other techniques are under the realm of Quality Assurance or Quality Control is a matter of extensive debate. In this chapter, we'll bring in various perspectives on this.

6.3 Structured Walkthroughs

6.3.1 What are Walkthroughs and what they can be used for

Structured Walkthroughs, which are also known as "team debugging", "peer code reviews" etc are some review methods that have received much attention. Structured walk through technique is particularly valuable in that it can be used throughout the software life cycle. For example, customer review is one of the most widespread and apparently effective methods used for evaluating the supporting plans of commercial software projects. Walkthroughs can in fact be held of virtually any work product e.g.:

- ☐ The work assignments and schedules
- ☐ Specifications
- ☐ Data structure designs
- ☐ Program designs
- ☐ Documentation, including User Manuals
- ☐ Code
- ☐ Test plan, data and results
- ☐ Maintenance changes

They can therefore be used in the early stages of software development, such as design and planning, i.e. long before testing can begin. A variety of definitions can be given for walkthroughs

- ☐ A formalization of free discussion
- ☐ A peer group review of any work product
- ☐ A review session in which the originator of a work product explains it to his/her colleagues, including the intended function of the product and the reason for the method chosen for preparing/developing the work product.
- ☐ An in-depth technical review of some aspect of a software system

Walkthrough are therefore a static method of QA. This is also a view held by Myers, Ref [8] according to whom Walkthroughs are of equal or even more value in checking modifications to programs, as modifying an existing program or a more error-prone process than writing a new program. Walkthrough are informal meetings but with a purpose. In a way, they are similar to Japanese quality circles (see the vignette below on "Quality Circles").

Quality Circles

Quality Circles are groups of 4-12 people from the same work group or from within a department or sometimes from cross-departmental boundaries. The objective of the individuals united in a 'Quality Circle' is to identify local problems and recommend solutions. They meet on a voluntary basis but regularly to discuss, identify, investigate, analyze and solve their own work-related problems.

• In some ways walkthroughs are similar to inspections (and hence the two terms are used interchangeably by some people) and yet in some other respects, they are different as will be discussed later in this chapter.

6.3.2 Walkthroughs—the objective and modus-operandi

As mentioned earlier, "Walkthrough" is one of the review methods and their objective is to ensure high quality, i.e. to find:

- ☐ Bugs, misinterpretations, omissions, inconsistencies, ambiguities, and anything that is unclear
- ☐ Anything that is complex or difficult to modify
- ☐ Deviation from standards (for simplicity, clarity, modularity, structure, documentation, external interfaces, user dialogues, exception conditions and exception handling)

It should be noted that the objective is to *find* problems and *not* correct them, because corrections are assumed to be within the province and capabilities of the developer.

Walkthroughs can be used with any of the various alternative team structures, but fit more naturally into the matrix structure because of its more democratic nature. They are less likely to be used in a rigid, bureaucratic organization. (a 'Matrix' structure is an attempt to combine the advantages of pure functional structure and the projectized organization structure)

Different authorities give different recommendations for the optimum number of participants in a walkthrough - about four persons participating in the walkthrough seems to be the generally held view. This is because, the more people there are, the greater the possibility of time-wasting due to differences of opinion, but more errors are likely to be uncovered. There are differences of opinions over *who* should attend walkthroughs, and it also depends on what type of work product e.g. specifications or code, is being walked through.

There is general agreement that the author of the product, a maintenance expert, and a member of the QA group should attend. There are both advantages and disadvantages if a manager attends a walkthrough. These are discussed later. Users (or customers) could attend specifications walkthroughs, testing walkthroughs, and possibly design walkthroughs, but not code walkthroughs. If users attend walkthroughs, designers may benefit from the increased involvement, but the users may use the walkthrough to ask for major changes. Other suggested participants include a technical secretary, a highly experienced programmer.

6.4 Inspections

Inspection as a review method was devised by Michael Fagan in IBM in 1972. As a technique they are more recent than "Walkthroughs". They are similar to walkthroughs but, there is an evidence that they are more effective. The ANSI/IEEE Std. 729-1983 IEEE Standard Glossary of Software Engineering Terminology defines inspection as

'..... formal evaluation technique in which software requirements, design or code are examined in details by a person or group other than the author to detect faults, violations of development standards, and other problems....'

According to Tom Gilb Ref [3] it may be wasteful to do Walkthroughs unless a document has successfully exited from Inspection. Thus, Inspection is not an alternative to Walkthroughs. In some cases it is a pre-requisite. These different review techniques have different purposes. Figure 1 shows the Inspection Process.

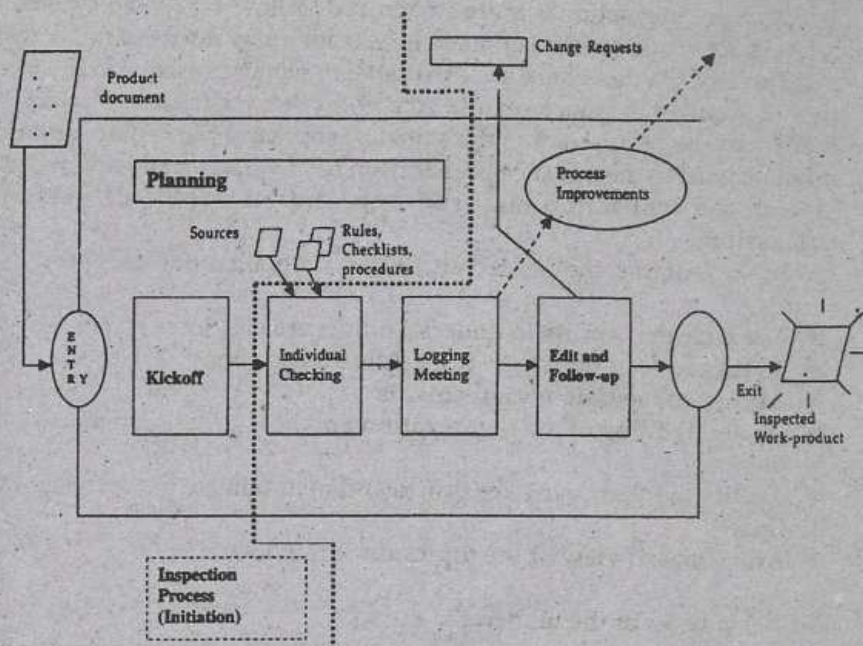


Figure 1: The Inspection Process

The various roles and responsibilities involved in Reviews and Inspections are next discussed.

6.5 Various Roles and Responsibilities Involved in Reviews/ Inspections

Watts Humphrey in his book Ref [1] has talked about the various participants in the inspection meeting :

- The MODERATOR (or INSPECTION LEADER)
- The PRODUCER (or the AUTHOR)
- The REVIEWER (or the INSPECTOR)
- The RECORDER (or the SCRIBE)

Let us now understand the roles and responsibilities of these various persons involved in inspections. Of all these roles mentioned above, the *Moderator* has a key role in the Inspection Process. A capable moderator will generally produce an effective inspection, while an inexperienced or poorly qualified one often will not do so. An inspection is an interpersonal activity and the moderator's role is to ensure that the people constructively interact to produce the proper result.

The *Moderator* (also called sometimes the '*Inspection Leader*') has the same responsibilities as in any formal technical review (FTR), but the emphasis may be more on interpersonal skills than technical skills.

Because the Inspection is more structured (following a fixed agenda of what is the scope on inspection etc) the leader may not need much technical ability. On the other hand, an inspection may consist of many more people than an ordinary review, so a good deal more conflict management may be required. For the same reason, very particular attention must be paid to the advance preparation of physical arrangements; otherwise the inspection may run aground on details of personal discussions.

In view of this, the desired attributes of a good moderator are:

- The technical ability to understand the area under review
- Ability to lead the group in effective discussions
- Ability to mediate disagreements
- Understanding of the organization and how to assign responsibilities
- Ability to identify the key issues and maintain the group's focus on them
- An unbiased view of the topics under review

The basic tasks of the moderator are to:

1. Obtain a good inspection or report to management whenever inspections could not be held along with the reasons.
2. Assist the producer in selecting the reviews and arranging for their participation.
3. Ensure that the reviewers do not have other reviews scheduled for the same day or have any other involvement that would impair their objectivity.
4. Conduct a pre-inspection session for all participants and ensure that they understand both their responsibilities and the inspection process.
5. Ensure that the entry criteria for inspection are met (Refer Figure 1).
6. Make sure that the scheduled participants are present and that no managers or observers are in attendance.
7. At the opening of the inspection, ensure that all participants are prepared or reschedule the inspection.
8. Conduct the inspection in an orderly and efficient manner.
9. Ensure that the inspection starts and ends on time.
10. Ensure that all identified problems are recorded and resolution responsibility is assigned.
11. Track each problem to resolution or ensure that it is tracked by someone else.
12. Gather the required inspection data and enter it into the process database.
13. Communicate the inspection results to all interested parties.

The *Producer* (alternatively called '*Author*') is the one with whom the entire review is aimed. The only reward for the Reviewers, the Moderator and the Recorder is the satisfaction of helping someone do a better job and the implied promise that they will be helped in return. In spite of the critical nature of the inspection process, the producers must remember that all these people are giving their valuable time to help them. The Producer's responsibilities are thus:

1. To ensure that the work is ready to be reviewed.
2. To make the preparatory material available on time for all the participants.
3. To support the Moderator in making the meeting arrangements, providing copies for the inspection package, and helping establish schedules.
4. To promptly resolve all the identified issues.
5. To check with the participants whenever issues are not clear or when ideas and suggestions have been offered.
6. To be objective and avoid becoming defensive.

This may sound very simple but it is not. It is probably the most difficult responsibility of all. After working for months on a task the producers will have already thought through and addressed many of the issues. It will therefore seem like an enormous waste of time explaining all over again. This, however, is a necessary cost of conducting an inspection, and it invariably pays enormous dividends.

The *Reviewers* (also known as the *Inspectors*), comparatively, have the easiest job in some respects. They need only focus on the technical issues, and, when the inspection is over their work is done. The Reviewers' particular responsibilities are:

1. Be prepared for the inspection.
2. Be objective; focus on issues and not on the people.
3. Concentrate on the problems, and offer suggestions on style or problem solutions before and after the inspection.
4. Address major issues and submit minor items separately.
5. Insist on understanding issues and proposed explanations. When something is not clear, do not hesitate to stop progress until you understand.
6. When you are shown to be wrong, forget it.
7. Support the moderator.
8. Do not hesitate to praise good work.

The Recorder's (or the Scribe) responsibility includes:

1. Being generally aware of the subject matter being reviewed.
2. Recording all issues raised during the Inspection and ensuring that persons raising them agree with the way they are recorded.
3. Recording any required data on the inspection and its preparation and provide it to the moderator.
4. Producing the final inspection reporting, listing all issues, the responsible party for resolving each, and the schedule for resolution.

At the end of this chapter various checklists are provided in relation to the Inspection process.

6.6 Some Psychological Aspects of Reviews

6.6.1 Why is it Tough to Get Reviewers?

"Cost of (Poor) Quality" is the combined cost of *Prevention*, *Appraisal* and *Failure* (Figure 1 Chapter 1). Efforts spent in "Reviews" constitute "appraisal cost". Yet, in practice, it is found that instituting a review culture is not at all easy. Karl Wiegers in his article [9] provides a very interesting account of how successful reviews are a matter of Culture, Attitude, and Expression. In this section we explain the (important but often ignored) psychological aspects of conducting reviews.

In the beginning of this chapter, it was said "to err is human...." And yet, a tremendous amount of resistance is faced in making review as the regular appraisal practice. Most of the objections seem to stem from psychological reasons as will be discussed in this section. After all, asking your colleagues to point out errors in your work is a "learned"—not instinctive behaviour as some of us may think. Most of us, as human beings do not like to admit that we make mistakes! We don't like to ask other people to find them. It is important to appreciate that holding successful peer reviews (an informal review carried out by colleagues or associates who do not have reporting relationship to the author/producer) requires us to overcome this natural resistance to outside critique of our work. In a healthy software engineering culture, team members engage their peers to improve the quality of their work and increase their productivity. They understand that time spent looking at a colleague's work product/deliverable isn't time wasted, especially when other team members willingly reciprocate. In lower maturity organizations, on the other hand, "busy" practitioners are mostly reluctant to spend time examining a colleague's work. The attitude is "Anyone who needs his/her code reviewed shouldn't be getting paid as a software developer". Thus, it is often, lack of knowledge about the review process, review techniques and cultural issues and simple resistance to change that keep organization from implementing reviews as a sound appraisal mechanism.

6.6.2 Tips for the Reviewer

The dynamics between the reviewers and the author of the work product are critical aspects of peer reviews. Mutual respect is involved – the author must trust and respect the reviewers in order to be receptive to their comments. Conversely, the reviewers must respect the author's talent and efforts put in. Reviewers must focus on the work product under review and not on the producer. A review is considered as failed if the review process is not properly controlled/facilitated—as a result of which author could walk out of the review meeting frustrated. It is important equally to understand that reviews are not meant to identify scapegoats for quality problems. Having a co-worker locate a defect is regarded as a "good catch" that could prevent a fatal defect, not a personal failing. Language and expression matter too. Reviewers should keep their focus objectively on the work product being reviewed; carefully selecting the words they use to raise an issue. For example, saying "I didn't see where these variables were initialized" is likely to get a far better response than saying "how on earth you forgot to initialize the variables..? You forget such simple things...and you've been programming for how many years now...." Avoid expressions that may sound like personal attacks. As some more examples, you could raise issues by saying "Are you sure that another component doesn't already provide that service?" or identify a point of confusion: "I didn't see where this memory block is de-allocated. Thus, to reiterate an important point—direct your comments to the work product, not to the author.

6.6.3 Tips for the Author/Producer

Cultural biases run deeper than workplace attitudes. For instance, our educational system grades people primarily on their individual performance, so collaborative work is sometimes viewed as "cheating". There is an implication that if you need help, you must not be very smart. This shuns many people from asking for a formal review of their work product. We have to overcome the ingrained culture of individual achievement and embrace the value of collaboration. Then there are the excuses that people will give to avoid review of their product. This has a cascading effect—because they have "avoided" review of their work products, chances are that most likely they will not participate in reviews of work product turned out by their co-workers! Or, on the other hand, there can be "I scratch your back and you scratch mine" (reviews not done honestly for vested interests!) People who don't want to do reviews will expend considerable amount of time and energy in explaining why reviews do not fit their culture, needs or time constraints. There is the attitude that some people's work does not need reviewing (as much as some reviewers can think—"so and so is brilliant programmer....who am I to look for errors in her work?). The

Table 1: Benefits of Review

<i>Project role</i>	<i>Possible benefits from peer reviews</i>
Developer	• Less time spent performing rework • Increased programming productivity • Better techniques learnt from other developers • Reduced Unit Testing and Debugging time • Less debugging during integration and system testing • Exchanging of information about components and overall system with other team members
Project Manager	• Shortened product development cycle time • Increased chance of shipping the product on schedule • Reduced field service and customer support costs • Reduced lifetime maintenance costs, freeing resources for new development projects • Improved teamwork, collaboration, and development effectiveness • Reduced impact from staff turnover through cross-training of team members
Product Maintainer	• Fewer production support demands, leading to a reduced maintenance backlog • More robust designs that tolerate change • Conformance of work products to team standards • Better understanding of the product from having participated in design and code reviews during development
Quality Assurance Manager or Quality Control Manager	• Ability to judge testability of product features under development • Shortened system testing cycles and less re-testing • Ability to use review data when making release decisions • Education of quality engineers about the product • Ability to participate quality assurance effort needed
Requirements Analyst	• Earlier correction of missing or erroneous requirements • Fewer infeasible and untestable requirements because of developer and test engineer input during reviews
Test Engineer	• Ability to focus on finding subtle defects because product is of higher initial quality • Fewer defects that block continued testing • Improved test design and test cases that smooth out the testing process

other typically heard excuse is "I'm too busy fixing my own bugs to waste time finding someone else's." Other developers imagine that their software ability has moved them past the point of peer reviews. They think "Inspections have been around for twenty-five years; they're obsolete" or "Inspections work in factory/manufacturing environment; not here—software is "different"

6.6.4 Overcoming the Resistance to Reviews

It is said that you hire a person for his/her attitudes. Skills can be imparted through training but attitudes are hard to change. Lack of knowledge is easy to correct if people are willing to learn. Review workshops, wherein hands-on exercise on performing review is given, help the participants. Management attendance in these workshop helps to spread the right message ("This is important enough for me to spend time on it, so it should be important for you too" and "I want to understand reviews so I can help make this effort succeed"). Dealing with cultural issues requires that you understand your team's culture and how best to steer the team members towards improved software engineering practices.

The table 1 on the previous page shows the benefits from peer reviews to various roles in a project.

6.7 Making Reviews and Inspections Effective

According to Watts Humphrey Ref [7], there are four reasons for inspections being effective: they provide an opportunity to look at the entire work product in one go, they make use of combined knowledge and group expertise, they take advantage of different viewpoints, and they improve the odd of finding problems. In the following sections we discuss these points.

6.7.1 Inspecting the entire Work product

The main reason why review techniques like Inspections, Walkthroughs are more effective than Testing is that in an inspection or a walkthrough; you see all the cases, combinations and conditions at once (for example in case of code). With testing however, you can try only one case, with one set of data values, and under one set of conditions. Thus, with an inspection or a walkthrough you can see whether any necessary conditions are missing or whether conditions overlap or conflict. With testing, on the other hand, you must run at least a few number of tests and even then you still cannot be sure that you have covered all the potential problems. Thus, with inspections and walkthroughs or desk checks, you can look for things that are not there, whereas, with testing you test only what is there and test it only incompletely (i.e. no guarantee on completeness of testing).

6.7.2 Using Combined Knowledge

When several people study a single work product, they can focus their combined knowledge on finding the problems. The engineers on an inspection team usually have broader base of knowledge and experience than the authors of the product alone. Examples of problems that commonly surface in an inspection are interface conflicts, misunderstood dependencies, and errors in naming convention. Most of these problems are found because the inspection team as a group knows more about these topics than the author does.

6.7.3 Using Different Viewpoints

Often, developers can get into a kind of myopia; it is a sort of human psychology—we see what we know should be there and tend to overlook obvious things that conflict with our expectations (cognitive dissonance). Thus, we develop a kind of blindness to our own mistakes. Involving a number of experts to review your work product have a number of benefits; not only will they have different interests, perspectives, and varied backgrounds, but also they will provide another benefits; as you listen to your own explanation of what you were trying to do, problems that you overlooked will often become obvious. When explaining your work, you look at it from a different perspective, and often this is all it takes to help you see previously overlooked problems.

6.7.4 Improving the Chances for Finding Errors

When several people review the same material, even if each of them finds only a fraction of the problems, their combined reviews often find many more of the problems than any one inspector could possibly find alone. Of course, defects are not random, and some defects are much more difficult to find than others. But, particularly with hard-to-find problems, it is important to have multiple reviewers. The chance of someone finding the truly difficult problems will then be substantially better. It appears that inspection yields also decline with higher defect densities. Inspection yield measures the percentage of the defects that were in the product at the beginning of the inspection and were found in the inspection. For effective inspections, therefore, the authors must first personally desk-check their products and remove all the obvious defects. The reason is that simple defects are distracting. When reviewers see trivial errors, they are likely to concentrate on them and are less likely to see more difficult problems. Although inspections take time, effective inspection involves more than just spending enough time; you must spend that time properly. This is question of well-defined inspection methods, use of checklists and various players being well aware of

their roles (The Reviewers/Inspector, The Author/Reviewee, The Moderator/Inspection Leader, the Records/Scribe as described in section 6.5). The general inspection methods are Checklists, Viewpoints and Product Concentration. Section 6.9 has discussed checklists. For discussion on Viewpoints and Product Concentration, reader are referred to [7]

6.8 Comparison of Review Techniques

Now that we've discussed various review techniques (Desk-checks, Walkthroughs and Inspections) one may wonder "which technique to use". A typically asked question is: Why do so many different people use so many different variations on the idea of Reviews? We attempt here to provide an answer to this often asked question as follows. The practice of (technical) review differs from place to place for a variety of reasons, the main ones being:

1. Different external requirements as dictated by various types of contracts
2. Different internal organizations, such as the use or nonuse of teams
3. Continuity with past practices that proved beneficial
4. The nature of work product under review itself could vary; for example reviews could be required of
 - a. Functional Specification Reviews
 - b. Design Reviews
 - c. Code Reviews
 - d. Documentation Reviews
 - e. Test Plan Reviews
 - f. Tool and Package Reviews
 - g. Reviews of Training Materials and Plans
 - h. Reviews of Procedures and Standards
 - i. Operations and Maintenance Reviews

Continuity is probably the strongest reasons in most situations. When it comes to social behavior, people tend to be very conservative about changing what they already know or have done. There is nothing wrong with establishing continuity with past practices as long as the underlying task of the review is accomplished. If an organization generally has well-led, productive meetings that are regarded by everyone involved, as worth their time, it's a good idea to associate the formal technical review with this tradition. On the other hand, in an organization, where people are generally allergic to "meetings", it may prove better strategy to emphasize the formal technical review as something different from past practices.

Various experts provide their perspectives on which review technique to use. Tom Glib [3] gives a golden rule: Use Walkthrough for

training, but use Inspection to improve the quality of the work product and its process. Traditionally, technical reviews and "structured walkthroughs" have been held for software development products. They have been found to be helpful in finding problems and improving the quality of software. However, these review techniques are much less formal than Inspection and much less effective at identifying according to Tom Glib [3]. They have been replaced by formal Inspection. Desk Check is an individual review technique whereas Walkthroughs are typically peer group discussion activities. Expected benefits of Inspection, when compared with other review techniques (walkthroughs and desk checks etc) are:

- Measurably high *product quality*
- Greater *productivity* from people in development and maintenance
- Shorter and more predictable *development times*

According to Freedman and Weinberg [4] "Inspection" approach tends to focus on a much narrower, much more sharply defined, set of questions. In some cases, an Inspection consists of running through a checklist of anticipated defects one after the other, over the entire work product (towards the end of this chapter, we have provided a couple of checklists.) Another way to try to cover more material is by having the product "walked through" by someone who is very familiar with it – even specially prepared with a more or less formal presentation. Walking through the product, a lot of detail can be skipped – which is good if you're just trying to verify an overall approach or bad if your objective is to find errors in detail.

In some cases, the Walkthrough is very close to a "lecture" about the product—which suggests another reason for varying the formal review approach. Reviews have educational qualities i.e. as Tom Gilb has said; Walkthroughs help towards training of new reviewers. This is a by-product of the quality assurance function. Sometimes, rapid education of large numbers of people may suggest some variation of the formal technical review. However, there may be situations where formality of reviews does not seem appropriate. For example, when members of the same team are reviewing each other's rough work (peer reviews), prior to finishing it for formal, external review, an informal version of the formal procedures may be comfortable. But there is one thing to watch against; as the review procedure gets less formal, however, there is a danger of drifting away from its central purpose. Another dimension for variation is the review team structure. At the one extreme, each review has a unique group chosen for it alone, but there are many variants that provide for some degree of permanence by having a group of people perform related reviews. At the other extreme, there are permanent review teams, devoted to specific work product, class of work product, or type of review.

IBM claims that Inspection give markedly greater productivity than Structured Walkthroughs, and result in fewer errors. Fagan listed the advantages of Inspection as:

- Inspectors have definite role to play;
- The moderator rather than the reviewee (the one whose work is under review) directs the effort .
- An error checklist is used; and
- Detailed error feedback is provided to individual programmers .

Inspections do not involve the members of the team which produced the work product being inspected, and so do not give the benefits of improved team communications, morale, and education of new members, provided by Walkthroughs. The key difference between Inspections and Walkthroughs is that Inspections are solely intended to detect defects. They do not have an explicit educational function, nor are they part of the design process where strategy decisions are made.

6.9 Inspection Related Checklists

Here are some useful checklists relating to Inspection as a review technique—they are:

- Checklist for the use by Inspection Moderator (Inspection Leader)
- Checklist for the Recorder (the Scribe)
- Checklists for inspection of documentation

These are presented below one by one.

Checklist for Inspection Moderator (The Review Leader)

Qualifications of the Moderator

- Do you understand the purpose of the inspections in general?
- Do you understand why this particular inspection is held?
- Can you be objective on the subject of the inspection?
- Have you ever participated in an inspection as reviewer or reviewee?
- Do you have any personal difficulties with any of the reviewers that might interfere with your ability to lead the inspection objectively?

Pre-Inspection

- Is the product ready for inspection?
- Are all relevant materials in your possession?
- Have all relevant materials been distributed on time?
- Have all the reviewers received the material?
- Have the reviewers confirmed their acceptance of the schedule?

- Has the suitable venue been booked for conducting the inspection?
- Have arrangement been made for the necessary equipment?
- During the Inspection
 - Are all participants well prepared?
 - Is there agreement on the objectives of the inspection?
 - Are all the participants contributing?
 - Is the inspection well paced?
 - Is interest waning as the time is progressing?
 - Is everyone being heard?
 - Is anyone being tuned out?
- Has someone (such as the producer/author) influenced/swayed participants with emotional arguments or other undue tactics?
- Is there agreement on the outcome of the inspection?
- Is that agreement truly understood by all participants?

Post-Inspection

- Was the inspection successful?
- Did the inspection reach a workable conclusion?
- Was anybody responsible, if the inspection was not successful?
- Is the report prompt and accurate?
- Are all participants satisfied with the outcome?
- Did the product get a fair and adequate treatment?
- Does the producing group have a reasonable basis for clearing up the issues?
- Have all relevant people received the appropriate information?
- What can be done to improve the inspection process to make it better next time?
- Has the required inspection data been gathered and recorded?
- Have all identified problems been resolved or informed to appropriate management level?

Checklist for Recorders (The Scribe)

Qualifications of the Recorder

- Do you understand the purpose of inspections in general?
- Do you understand why this particular inspection is being held?
- Do you understand the jargon used and the formats used in this material?
- Are you able to communicate with the type of people who will be in the inspection?
- Have you ever participated in an inspection as a reviewer or as a reviewee?

Pre-Inspection

- Can you identify, by name, the inspection leader and other participants?

Have you arranged your schedule to allow time for the inspection?
Have you allowed time for the work you will have to do after the inspection?

Do you have the materials necessary for keeping an accurate record in the proper formats?

Do you have the resources available to carry out your job, during and after the inspection?

During the Inspection

Do you understand what an issue is?

Are you recording all issues?

Are you recording things that aren't really issues?

Are your notes accurate reflections of the comments received during the inspection?

Do you have copies of any supplementary material introduced as a part of any issue?

How much of your report consists of editorial commentary?

Is the outcome stated explicitly and unambiguously?

Are the issues recorded in neutral language?

Post-Inspection

Was the report promptly prepared?

Was the report accurate?

Was the report properly reviewed and signed?

Was it distributed to all relevant people including participants in the inspection?

Was all pertinent data on the inspection and its preparation gathered and provided to the moderator?

Conclusion

An inspection is a more formal review technique than Desk Check and Walkthrough. Overall, Inspections appear to provide a substantial increase in both development and particularly maintenance productivity, and also find many errors. However, since the results of Inspections (being a more formal review technique) are informed to management, people may feel threatened. There are some enthusiastic proponents of Inspections. Although Walkthroughs and Inspections have some common aspects, a disadvantage is that the Inspection process is dependent on the creation and use of less ambiguous design specification languages.

REFERENCES

- [1] *Managing the Software Process*—Watts Humphrey (Addison Wesley)
- [2] *Software Engineering: A Practitioner's Approach*—Roger Pressman (McGraw Hill International)